

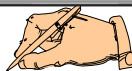
REPUBLIQUE DEMOCRATIQUE DU CONGO
INSTITUT SUPERIEUR PEDAGOGIQUE



SECTION DES SCIENCES EXACTES
Départements de Mathématiques et d'Informatique

**ALGORITHMIQUE & METHODES DE
PROGRAMMATION**

SUPPORT DE COURS DESTINE AUX ETUDIANTS DE PREMIERE
ANNEE EN INFORMATIQUE



Ruffin-Benoît NGOIE MPOY
Docteur en Mathématiques Appliquées

ANNÉE ACADÉMIQUE 2019 – 2020

Chapitre 1 : Généralités sur l'Algorithmique

1.1. Introduction

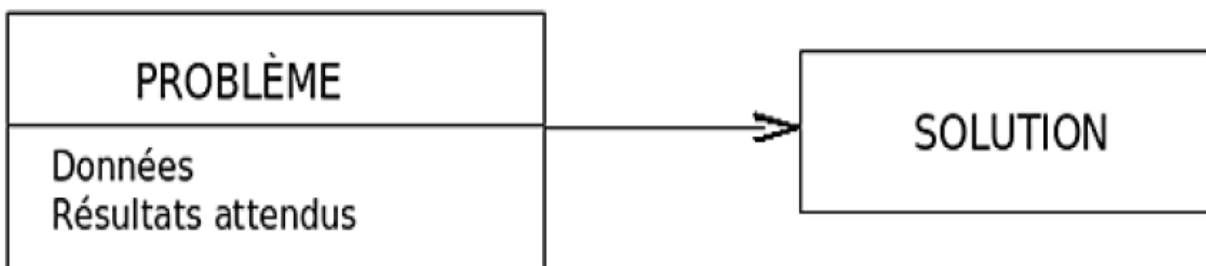
L'objectif principal de l'Algorithmique est d'obtenir d'un ordinateur qu'il effectue un travail à la place de l'homme. Ainsi, se pose alors le problème de communication entre ce dernier et la machine : *Expliquer à la machine comment elle doit s'y prendre.*

L'enseignement du cours d'Algorithmique en première année a plusieurs objectifs, notamment :

- Résoudre des problèmes comme une machine ;
- Savoir expliciter son raisonnement ;
- Savoir formaliser son raisonnement ;
- Concevoir et écrire des méthodes de résolution des problèmes ;

1.2. Qu'est-ce-que c'est que l'Algorithmique ?

On s'intéresse à l'activité de programmation d'un ordinateur qui permet de résoudre des problèmes d'une façon automatique. On se place à un niveau conceptuel dans lequel un problème quelconque, informatique ou non, est caractérisé par un ensemble de données d'entrée et des résultats attendus. On peut donc représenter ce niveau d'analyse par le schéma de la figure ci-dessous.



Exemple 1

On donne les ingrédients suivants :

Thym – laurier – persil – ail – huile – vinaigre – sel – poivre – citron et un brochet de trois livres. Préparer une marinade de brochet.

Dans cet exemple, le problème est représenté par les données : le thym, le laurier, ..., le brochet dont on connaît le poids et par le résultat attendu qui est un plat cuisiné : une marinade de brochet. Ce n'est pas (pas encore) un problème qui se pose habituellement en informatique, mais il se pose dans les mêmes termes.

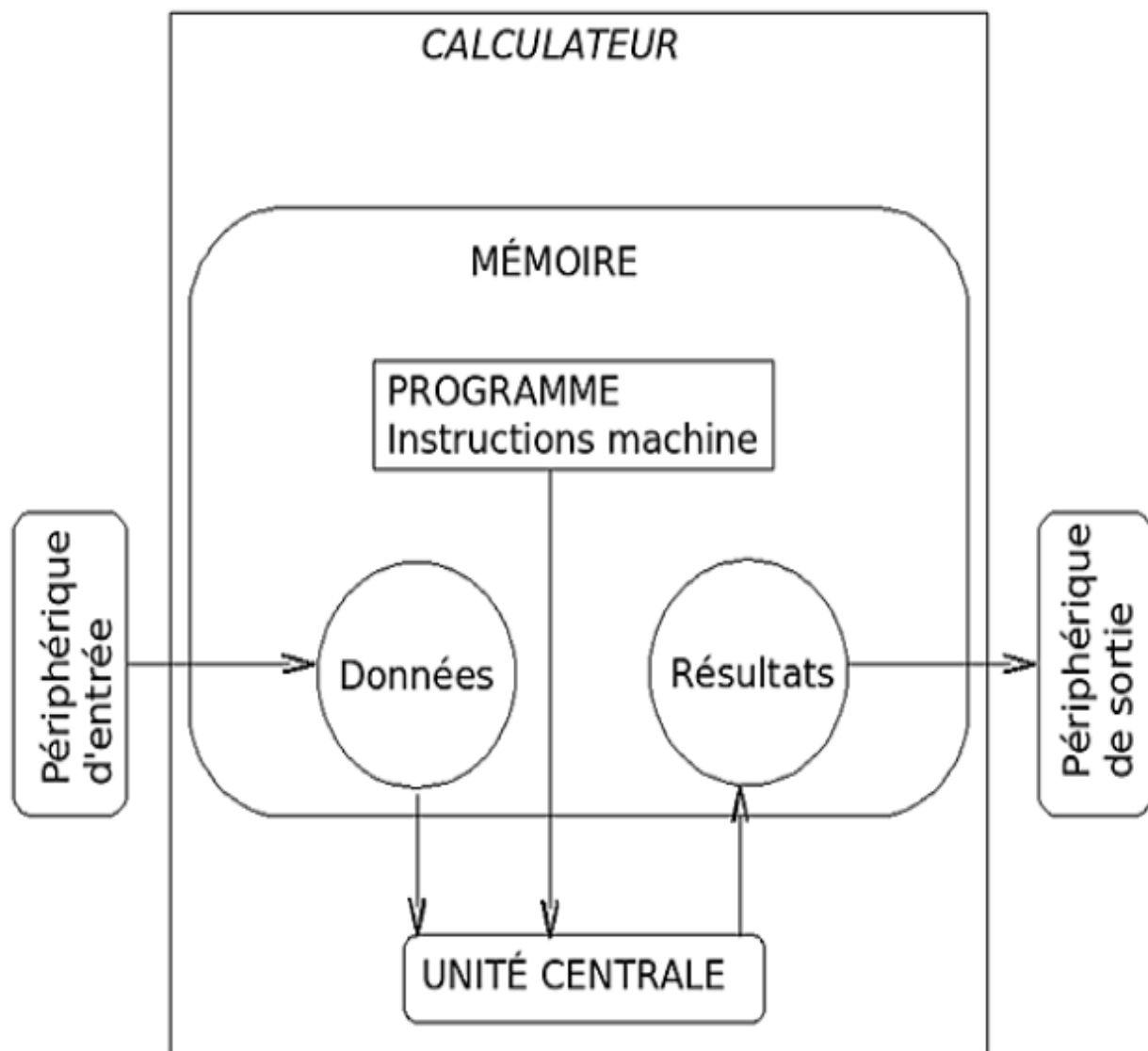
Exemple 2

On demande de calculer, pour une année dont on donne le millésime, le jour de la semaine où tombe le premier mai.

Ici, les données sont constituées d'un millésime, comme 2005 par exemple. Le résultat attendu est un jour de la semaine, comme dimanche par exemple.

Dans ces exemples, le problème est constitué d'un jeu de données. Le résultat attendu doit être obtenu par des transformations à faire subir aux données ; c'est ce que représente la figure ci-dessus.

Les problèmes auxquels on s'intéresse dans ce cours sont résolus à l'aide d'un ordinateur. Un ordinateur est constitué d'une machine matérielle complétée d'un ensemble de composants périphériques, comme un clavier, une souris, une table traçante... La machine matérielle est constituée, en première approximation, d'une unité centrale, chargée d'effectuer les calculs, complétée d'une mémoire, permettant d'enregistrer les données, les résultats des calculs intermédiaires et les résultats attendus. Pour résoudre un problème avec l'aide d'un ordinateur, il faut enregistrer dans sa mémoire, d'une part les données du problème, qu'il devra transformer pour obtenir le résultat attendu et, d'autre part, la suite des instructions que l'unité centrale du calculateur devra exécuter pour effectuer les transformations des données. Le résultat étant obtenu, il sera d'abord enregistré, comme les données, dans la mémoire. La figure suivante représente ces éléments :



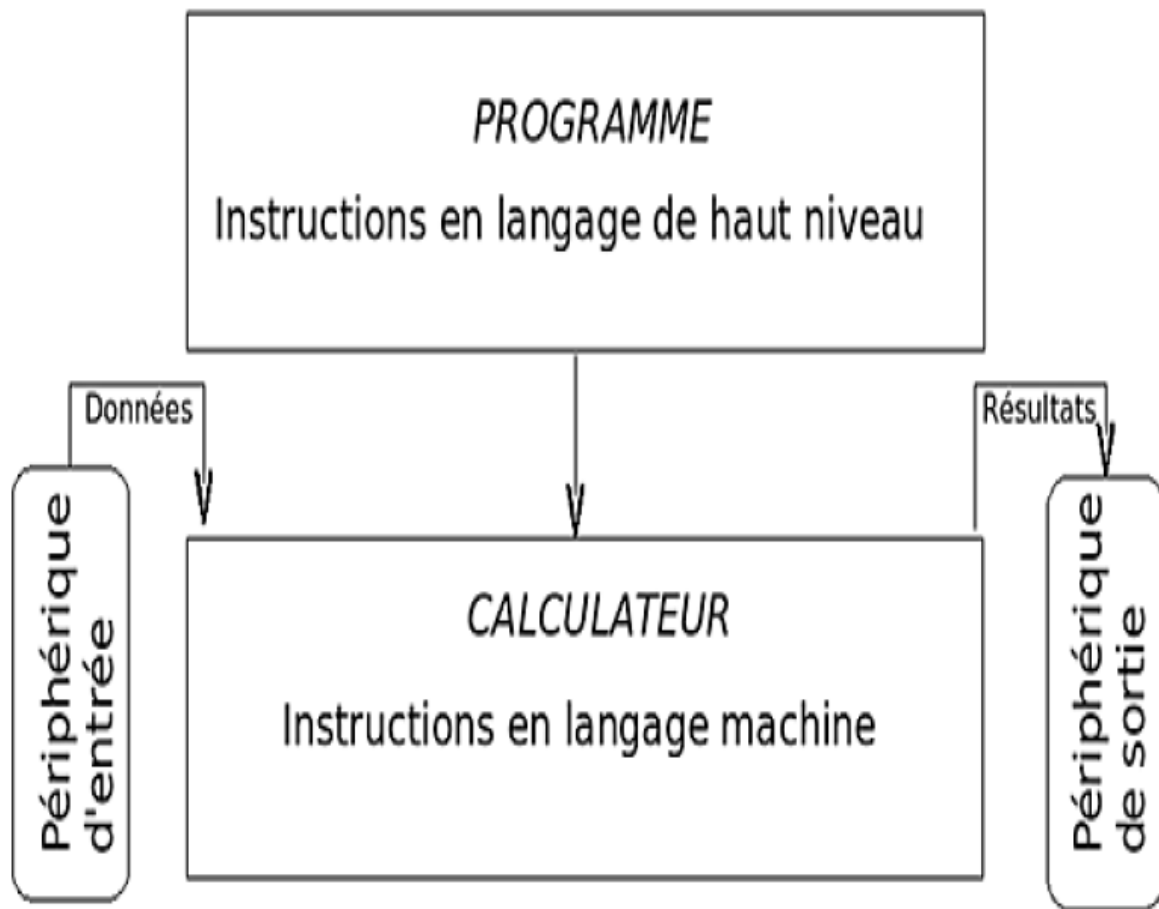
Un périphérique d'entrée permet de communiquer au programme les données à transformer. Un programme, constitué d'instructions en langage machine exécutées par l'unité centrale, les transforme et produit des résultats qui sont envoyés au périphérique de sortie.

Un premier élément, remarquable ici, est l'architecture mise en œuvre. Les instructions-machine exécutées par l'unité centrale et les données sur lesquelles elles agissent sont rangées et cohabitent dans la même mémoire centrale. Une telle architecture est celle des ordinateurs depuis leur tout début, dans la première moitié du vingtième siècle. Elle a été inventée et théorisée par John VON NEUMANN et Alan TURING. Dans cette architecture, l'unité centrale n'exécute qu'une seule

instruction à la fois et ces instructions sont exécutées séquentiellement, l'une après l'autre, dans un ordre préétabli et définitivement fixé. On dit qu'il s'agit d'une machine séquentielle pour caractériser un tel comportement. Il existe d'autres types de machines qui ne nous intéressent pas dans ce cours.

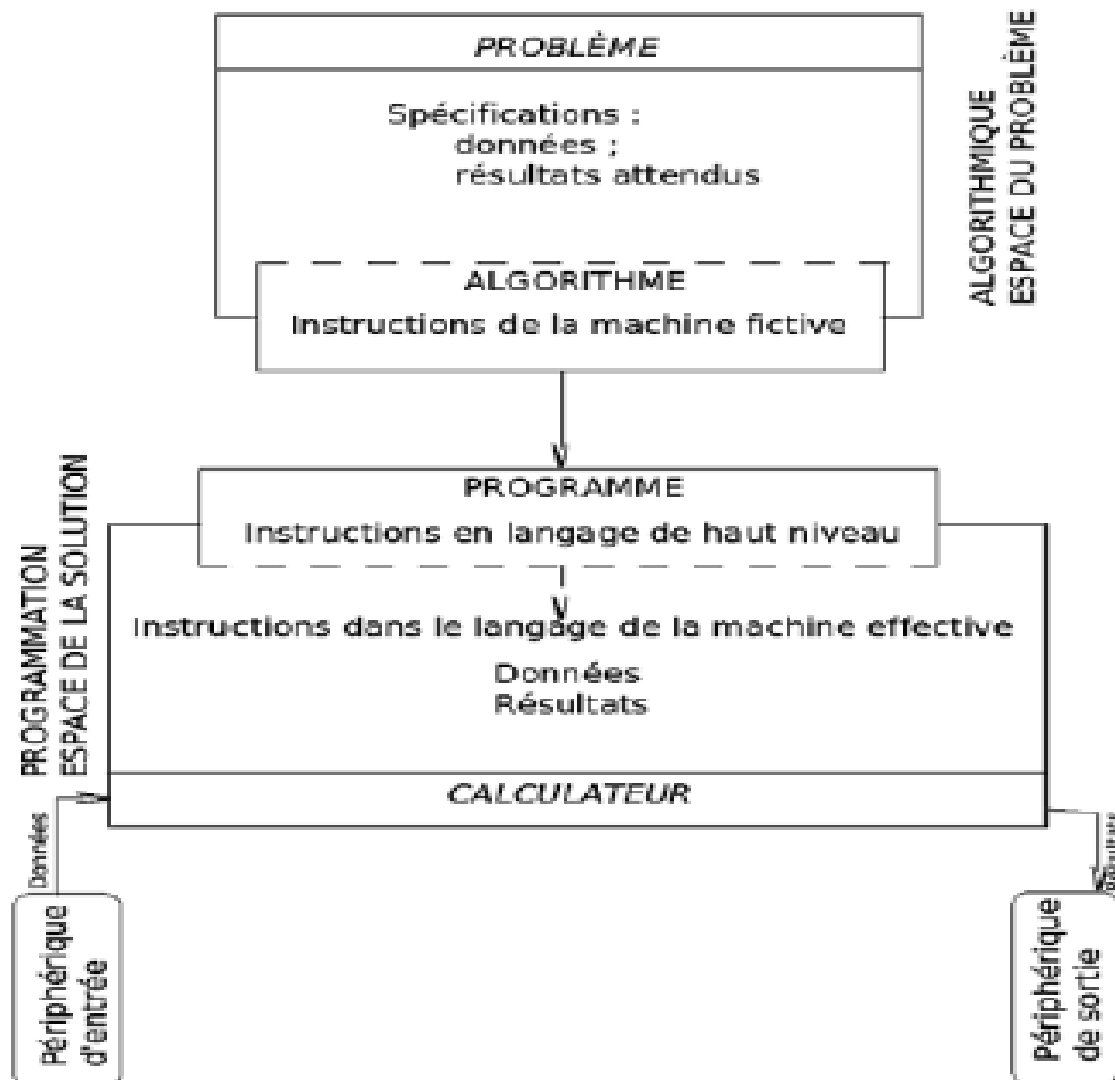
Une deuxième propriété importante de telles machines est que les mêmes données, communiquées au même programme, produisent les mêmes résultats, indépendamment de l'environnement et du moment de l'exécution des instructions. On dit alors que ce type de machine est déterministe. Là encore, ce n'est pas la seule architecture matérielle possible, mais ce cours se restreint à ce cas.

Pour obtenir un programme et les instructions qui le composent, on le prépare en utilisant un langage de programmation à l'aide duquel on écrit les instructions qui transforment les données. Il existe de nombreux langages de programmation de haut niveau, certains dédiés à des tâches spécifiques, comme les tableurs par exemple, d'autres destinés à écrire des programmes plus généraux, comme QBASIC, JAVA, PHP, C... La figure ci-dessous représente les relations entre un programme de haut niveau et le calculateur auquel il est destiné.



Pour passer des données d'un problème aux résultats attendus, on considère une machine fictive caractérisée par un comportement qui lui permet de réaliser les opérations. Cette machine fictive, comme son modèle, l'ordinateur de VON NEUMANN, n'effectue qu'une seule opération à la fois pour réaliser un traitement précisément défini. Les opérations s'organisent en une suite ordonnées d'instructions qui opèrent sur de l'information symbolique, c'est-à-dire sur une suite de signes alphabétiques, numériques, lexicaux. Cette machine fictive est donc une machine séquentielle, comme son modèle.

On appelle calcul une suite de traitements réalisés par cette machine, machine abstraite le calculateur fictif et programme abstrait la suite d'instructions préparées pour une telle machine. Cependant, alors que les programmes écrits dans un langage de programmation et destinés à un calculateur concret doivent respecter une syntaxe précise et rigide, les programmes abstraits ne sont contraints par aucune règle de « grammaire », aucune syntaxe imposée. En ce sens, un programme abstrait n'est pas un programme et n'utilise pas un langage différent du langage naturel. On appelle algorithme un programme écrit pour la machine abstraite. La figure ci-dessous représente les relations entre la machine abstraite et son modèle, le calculateur concret.



1.3. Problèmes fondamentaux en Algorithmique

1.3.1. Complexité

Cette science vise à déterminer, sous un certain nombre de conditions, quel sont les temps de calcul et espace mémoire requis pour obtenir un résultat donné. Une bonne maîtrise de la complexité se traduit par des applications qui tournent en un temps prévisible et sur un espace mémoire contrôlé.

Bref, l'objectif de la complexité est de répondre aux questions suivantes :

- En combien de temps un algorithme va-t-il atteindre le résultat escompté ?
- De quelle quantité de mémoire a-t-il besoin ?

1.3.2. Calculabilité

Elle vise à répondre aux questions que voici :

- Existe-t-il des tâches pour lesquelles il n'existe aucun algorithme ?
- Etant donné une tâche, peut-on dire s'il existe un algorithme qui la résolve ?

1.3.3. Correction

Cette branche permet de s'assurer qu'un algorithme répond au problème pour lequel il a été conçu.

1.4. Notion d'Algorithme

1.4.1. Qu'est-ce qu'un algorithme ?

Les mathématiciens définissent un algorithme comme une méthode de résolution d'un problème suivant un enchaînement déterminé de règles opératoires.

En informatique, nous le définissons comme un procédé mis en œuvre sur un ordinateur, et qui, répété autant de fois qu'il est nécessaire, permet d'obtenir le résultat cherché.

Le mot algorithme n'est pas dérivé d'un mot latin ou grec, mais d'une contraction et d'une dérivation du nom du mathématicien arabe Al – Khwarizmi qui publia deux livres importants : l'un sur l'arithmétique et l'autre sur « l'action de faire passer et d'agencer les parties d'un tout » ⁽¹⁾

Trois siècles plus tard, le livre, traduit en latin, porta le nom « Algorismus ».

La première définition du mot algorithme dans son sens actuel a été donnée par le mathématicien russe Markov : « *Tout ensemble de règles précises destiné à obtenir un résultat déterminé à partir de certaines données initiales* »

¹ Le titre original de cet ouvrage est « Kital Al-jabr wal muqabala »

Plus simplement, on peut dire qu'un algorithme est la description précise, sous forme de concepts simples, de la manière dont on peut résoudre un problème.

Dans la vie de tous les jours, nous avons souvent besoin de résoudre des problèmes. Surtout si on considère la notion de "problème" au sens large.

Les algorithmes sont constitués d'un ensemble de règles précises et compréhensibles par tous. Ils s'appliquent à des données qui peuvent changer, élaborent des résultats en fonction des données initiales.

La réalisation d'un algorithme est un acte créatif : Il n'existe pas de méthode pour découvrir un algorithme.

1.4.2. Exemple d'un algorithme

Un exemple de problème qui nous concerne tous est celui de la cuisine : vous êtes dans une cuisine, vous trouvez du riz, comment le cuire ? Voici une marche à suivre simple :

1. remplir une casserole d'eau ;
2. y ajouter une pincée de sel ;
3. la mettre sur le feu ;
4. attendre l'ébullition de l'eau ;
5. mettre le riz dans la casserole ;
6. le laisser cuire 10 à 15 minutes ;
7. égoutter le riz.

Nous avons décrit une solution au problème "il faut faire cuire du riz", sous forme de concepts simples. Il y a cependant lieu de remarquer qu'il y a beaucoup de choses implicites : Nous avons précisé que vous étiez au départ en possession du riz, mais il faut aussi une casserole, de l'eau, etc. On peut se trouver dans des situations spécifiques où tous ces objets ne sont pas disponibles, et il faudra alors utiliser un autre algorithme (ou commencer par construire une casserole...).

Les instructions que nous avons utilisées sont "précises", mais on pourrait préciser moins de choses, ou plus. Comment fait-on pour remplir une casserole d'eau, plus précisément ? Si le cuisinier à qui la recette est destinée ne sait pas

interpréter la ligne "remplir une casserole d'eau", il faudra l'expliquer en termes plus simples (en expliquant comment utiliser le robinet, par exemple).

De même, quand vous programmez, le degré de précision que vous utilisez dépend de nombreux paramètres : le langage que vous utilisez, les bibliothèques que vous avez à disposition, etc.

1.4.3. Différences entre algorithmes et programmes

Pourquoi apprendre l'algorithmique pour apprendre à programmer ? En quoi a-t-on besoin d'un langage spécial, distinct des langages de programmation compréhensibles par les ordinateurs ?

Parce que l'algorithmique exprime les instructions résolvant un problème donné indépendamment des particularités de tel ou tel langage. Pour prendre une image, si un programme était une dissertation, l'algorithmique serait le plan, une fois mis de côté la rédaction et l'orthographe. Or, vous savez qu'il vaut mieux faire d'abord le plan et rédiger ensuite que l'inverse...

Apprendre l'algorithmique, c'est apprendre à manier la structure logique d'un programme informatique. Cette dimension est présente quel que soit le langage de programmation ; mais lorsqu'on programme dans un langage (en QBasic, en C, en Visual Basic, etc.) on doit en plus se collecter les problèmes de syntaxe, ou de types d'instructions, propres à ce langage. Apprendre l'algorithmique de manière séparée, c'est donc sérier les difficultés pour mieux les vaincre.

A cela, il faut ajouter que des générations de programmeurs, souvent autodidactes (mais malheureusement pas toujours), ayant directement appris à programmer dans tel ou tel langage, ne font pas mentalement clairement la différence entre ce qui relève de la structure logique générale de toute programmation (les règles fondamentales de l'algorithmique) et ce qui relève du langage particulier qu'ils ont appris. Ces programmeurs, non seulement ont beaucoup plus de mal à passer ensuite à un langage différent, mais encore écrivent bien souvent des programmes qui même s'ils sont justes, restent laborieux. Car on n'ignore pas impunément les règles fondamentales de l'algorithmique... Alors, autant l'apprendre en tant que telle!

Un **programme** est la réalisation (l'implémentation) d'un algorithme au moyen d'un langage donné (sur une architecture donnée). Il s'agit de la mise en œuvre du principe. Par exemple, lors de la programmation on s'occupera parfois explicitement de la gestion de la mémoire qui est un problème d'implémentation ignoré au niveau algorithmique.

1.4.4. Rôle privilégié des ordinateurs

Si on trouve des algorithmes dans la vie de tous les jours, pourquoi en parle-t-on principalement en informatique ? La raison est très simple : les ordinateurs sont très pratiques pour effectuer des tâches répétitives. Ils sont rapides, efficaces, et ne se lassent pas.

On peut décrire un algorithme permettant de calculer les décimales de la racine carrée de deux, qui soit utilisable par un humain. Vous pourrez ainsi calculer, à l'aide d'une feuille et d'un crayon, les 10 premières décimales (1,4142135624). Mais s'il vous en faut un million ? Un ordinateur deviendra alors beaucoup plus adapté.

De manière générale, on peut concevoir de nombreux algorithmes comme des méthodes de traitement d'information : recherche, comparaison, analyse, classement, extraction, les ordinateurs sont souvent très utiles pour trier la masse d'informations qui nous entoure continuellement.

Vous aurez peut-être pensé au célèbre moteur de recherche Google (qui a initialement dominé le marché grâce aux capacités de son algorithme de recherche), mais ce genre d'activités n'est pas restreint au (vaste) secteur d'Internet : quand vous jouez à un jeu de stratégie en temps réel, et que vous ordonnez à une unité de se déplacer, l'ordinateur a en sa possession plusieurs informations (la structure de la carte, le point de départ, le point d'arrivée) et il doit produire une nouvelle information : l'itinéraire que doit suivre l'unité.

1.4.5. Structures de données

En plus de manipuler l'information, il faut aussi la stocker. La manière dont on organise cette information stockée peut avoir des conséquences très importantes sur leur manipulation.

Concrètement, prenez par exemple un dictionnaire : on peut définir un dictionnaire comme "un ensemble de mots et leur définition". Cependant, pourrait-on utiliser correctement un dictionnaire dont les mots sont placés dans le désordre ? Certainement pas, parce qu'il serait très difficile de trouver la définition d'un mot que l'on cherche (c'est encore possible, il suffit de lire le dictionnaire page par page jusqu'à ce qu'on trouve le mot). L'ordre alphabétique est clairement une solution très efficace pour pouvoir retrouver rapidement le mot que l'on cherche.

Une structure de données est donc une description de l'organisation des données gérées par un algorithme.

Il y a des liens forts entre les algorithmes (qui décrivent des méthodes) et les structures de données (qui décrivent une organisation). Typiquement, certaines structures de données sont indispensables à la mise en place de certaines méthodes, et à l'inverse certains algorithmes sont nécessaires aux structures de données : par exemple, si on veut rajouter un mot dans un dictionnaire classé alphabétiquement, on ne peut pas juste l'écrire dans l'espace libre sur la dernière page, il faut utiliser un algorithme pour l'ajouter au bon endroit. L'étude des structures de données est donc inséparable de celle des algorithmes, et vous n'y échapperez pas.

1.4.6. Conventions en algorithmique

Historiquement, plusieurs types de notations ont représenté des algorithmes.

Il y a eu notamment une représentation graphique, avec des carrés, des losanges, etc. qu'on appelait des organigrammes. Aujourd'hui, cette représentation est quasiment abandonnée, pour deux raisons. D'abord, parce que dès que l'algorithme commence à grossir un peu, ce n'est plus pratique du tout du tout. Ensuite parce que cette représentation favorise le glissement vers un certain type de programmation, dite non structurée, que l'on tente au contraire d'éviter.

C'est pourquoi on utilise généralement une série de conventions appelée « pseudo-code », qui ressemble à un langage de programmation authentique dont on aurait évacué la plupart des problèmes de syntaxe. Ce pseudo-code est susceptible de varier légèrement d'un livre (ou d'un enseignant) à un autre. C'est bien normal : le pseudo-code, encore une fois, est purement conventionnel ; aucune machine n'est

censée le reconnaître. Donc, chaque cuisinier peut faire sa sauce à sa guise, avec ses petites épices bien à lui, sans que cela prête à conséquence.

1.4.7. Quelques conseils pour être bon en algorithmique

La maîtrise de l'algorithmique requiert deux qualités, très complémentaires d'ailleurs :

- il faut avoir une certaine *intuition*, car aucune recette ne permet de savoir a priori quelles instructions permettront d'obtenir le résultat voulu. C'est là, si l'on y tient, qu'intervient la forme « d'intelligence » requise pour l'algorithmique. Alors, c'est certain, il y a des gens qui possèdent au départ davantage cette intuition que les autres. Cependant, et j'insiste sur ce point, les réflexes, cela s'acquiert. Et ce qu'on appelle l'intuition n'est finalement que de l'expérience tellement répétée que le raisonnement, au départ laborieux, finit par devenir «*spontané*».
- il faut être *méthodique* et *rigoureux*. En effet, chaque fois qu'on écrit une série d'instructions qu'on croit justes, il faut systématiquement se mettre mentalement à la place de la machine qui va les exécuter, armé d'un papier et d'un crayon, afin de vérifier si le résultat obtenu est bien celui que l'on voulait. Cette opération ne requiert pas la moindre once d'intelligence. Mais elle reste néanmoins indispensable, si l'on ne veut pas écrire à l'aveuglette.

Et petit à petit, à force de pratique, vous verrez que vous pourrez faire de plus en plus souvent l'économie de cette dernière étape : l'expérience fera que vous « verrez » le résultat produit par vos instructions, au fur et à mesure que vous les écrirez. Naturellement, cet apprentissage est long, et demande des heures de travail patient. Aussi, dans un premier temps, évitez de sauter les étapes : la vérification méthodique, pas à pas, de chacun de vos algorithmes représente plus de la moitié du travail à accomplir... et le gage de vos progrès.

Comme dernier conseil, nous paraphraserons Napoléon Bonaparte : « N'attribuez jamais à la malveillance ce qui s'explique très bien par l'incompétence. »

Mais sachez aussi qu'à l'origine de toute erreur attribuée à l'ordinateur, vous trouverez au moins deux erreurs humaines. Dont celle consistant à attribuer l'erreur à l'ordinateur.

1.4.8. Sémantique et Syntaxe des algorithmes

La partie syntaxique indique les règles à utiliser pour ne pas commettre des *fautes d'orthographe* et la partie sémantique indique le sens associé à cette construction.

1.4.9. Caractéristiques d'un algorithme

L'algorithme est un moyen pour le programmeur de présenter son approche du problème à d'autres personnes. Un algorithme doit donc être :

- Lisible
- De haut niveau
- Précis
- Concis
- Structuré

1.4.10. Erreurs en programmation

Durant l'écriture d'un programme, on peut être confronté à deux types d'erreur :

- *Erreurs syntaxiques*

Ces erreurs se remarquent à la compilation et sont le résultat d'une mauvaise écriture dans le langage de programmation.

- *Erreurs sémantiques*

Ces erreurs se remarquent à l'exécution et sont le résultat d'une mauvaise analyse. Elles sont beaucoup plus graves car elles peuvent se déclencher en cours d'exploitation du programme.

1.5. Ordinogramme

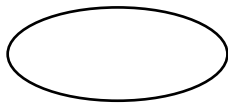
L'ordinogramme est la représentation graphique, en utilisant des signes conventionnels, d'un algorithme.

Il montre principalement les opérations et décisions nécessaires à l'exécution complète du processus. En d'autres termes, c'est une représentation schématisée par des graphiques et conduisant à la résolution d'un problème.

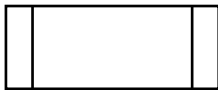
Alors, un ordinogramme est un schéma symbolique représentant les différentes phases de traitement ainsi que leurs relations logiques.

Le but d'un ordinogramme est d'exprimer et de permettre de suivre la ligne du raisonnement logique. Son but accessoire est d'être un lien entre la définition explicite du problème et les programmes des instructions. Autrement dit, il aide à transformer en un programme le raisonnement établi pour réaliser les objectifs donné du problème.

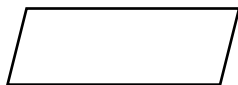
1.5.1. Symboles d'ordinogramme



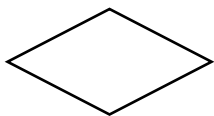
Début/Fin



Sous-programme



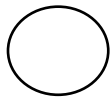
Lecture/Ecriture (Entrée/Sortie)



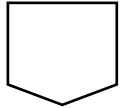
Test



Traitement



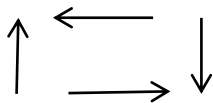
Branchement sur la page en cours



Branchement sur une autre page



Aiguillage



Sens logique d'exécution

1.5.2. Règles de graphisme

Quelques règles simples doivent être respectées pour obtenir un schéma clair permettant de résoudre un problème. Ces règles sont les suivantes :

1. On commence toujours un ordinogramme par le symbole de début à l'intérieur duquel on écrit le mot « Début » et on terminera l'ordinogramme par le même symbole mais avec le mot « Fin »
2. Il faut toujours entrer dans le symbole par le dessus
3. Les lignes de raccordement (branchement) sont verticales ou horizontales. Utilisées sur toutes les lignes, les flèches indiquent le sens de branchement.
4. Pour chaque symbole, il y a une seule entrée dans la partie supérieure et une sortie dans la partie inférieure sauf pour le symbole de test.
5. Il faut éviter l'intersection de deux lignes
6. Marquer l'intersection de deux lignes par un point
7. Marquer le passage d'un branchement sur un autre par une courbe
8. Mettre à l'intérieur de chaque symbole l'opération que l'on y effectue
9. Dans le symbole de traitement, on peut y mettre une ou plusieurs instructions.
10. Aucun symbole ne doit figurer sur une ligne horizontale
11. Ecrire est l'action provoquant l'écriture à l'écran d'une chaîne de caractères comprise entre les apostrophes

12. Lire est l'action qui attend de l'utilisateur une saisie au clavier
13. Le symbole $\leftarrow$ Signifie que la variable située à gauche du symbole prend pour valeur celle de l'expression située à droite du symbole.

Chapitre 2 : Démarche de résolution de problèmes

Parmi les programmes, nous citons les systèmes d'exploitation permettant comme leur nom l'indique, l'exploitation de l'ordinateur, ses divers périphériques ainsi que les différents programmes à faire exécuter. Certains modules des systèmes d'exploitation permettent la communication entre deux ou plusieurs ordinateurs, ils assurent la mise en marche d'un réseau informatique. Dans ces mêmes systèmes, nous avons aussi plusieurs autres applications permettant de répondre à divers besoins tels que le traitement de texte, le dessin, le son, la téléphonie et même les jeux. Dans notre quotidien, l'informatisation évolue d'une façon vertigineuse, en effet, tous les domaines de la vie courante ou presque, sont devenus informatisés. En partant du récepteur numérique d'un téléviseur et d'un téléphone portable jusqu'aux différentes factures qui nous parviennent, toutes ces opérations deviennent gérées par des programmes informatiques que les ordinateurs exécutent avec les plus grandes précision et rapidité.

Nous pouvons citer quelques exemples d'applications de l'informatique dans le quotidien que nous vivons :

- Dans un téléphone portable, il y a des programmes de jeux. De temps à autres, quand on s'ennuie, on lance son jeu préféré. Le portable devient un petit ordinateur !
- Dans la pharmacie du quartier, les agents saisissent les noms des médicaments ; l'ordinateur se charge du reste et à l'aide de l'imprimante, il inscrit le total à payer directement sur l'ordonnance.
- Dans une école, il y a un agent de l'administration qui gère avec un ordinateur les notes des élèves et fait sortir leurs bulletins pour chaque trimestre.

Toutes les pratiques informatisées et les exemples évoqués sont gérés par des programmes que l'ordinateur exécute pour résoudre un problème préalablement posé. Nous allons voir dans la suite et, étape par étape, la démarche à suivre pour résoudre un problème pose. Nous ne prétendons pas résoudre des problèmes similaires à ce que nous venons d'évoquer mais nous allons apprendre les bases d'une telle démarche.

2.1. Première étape : Position du problème

Le problème est souvent posé par un demandeur de solution informatique. C'est l'exemple du pharmacien qui veut gérer toutes les opérations de sa pharmacie, ou une société de transport qui veut superviser les réservations de ses clients ou encore le professeur qui compte informatiser son carnet de notes. Souvent les énoncés formulés par le poseur du problème manquent de précision, il est donc impératif à l'homme d'étude de reformuler en détail avec l'intéressé, le problème posé.

Exemples :

- Ecrire un programme qui calcule et affiche les moyennes des élèves d'une classe. Comme vous le constatez, cet énoncé manque de précision. En effet, il manque le nombre de notes, leurs coefficients respectifs, etc.
- On compte écrire un programme qui résout des équations du premier degré. On pourra se poser la question à propos du domaine des coefficients ainsi que celui de la résolution.
- Un pharmacien se propose d'informatiser les différentes opérations de sa pharmacie. Bien entendu, ce cas est plus complexe et suscite davantage de questions. Quelles sont les actions concernées par l'informatisation : la facturation, les commandes, la paie du personnel, etc. Cet exemple gère aussi des bases de données, donc il est important de savoir comment les concevoir et les construire.

Exercices

Donner une formulation détaillée pour chacun des problèmes suivants :

- Ecrire un programme qui résout une inéquation.
- Ecrire un programme qui calcule le bénéfice fait suite à la vente d'un produit.
- Ecrire un programme qui calcule le quotient de deux nombres.
- Ecrire un programme permettant d'informatiser l'édition d'une facture comme celle de la REGIDESO ou de la SNEL.

Bien entendu, si l'énoncé du problème est clair et précis, cette étape est gagnée.

Notons qu'à l'issue de cette étape, le problème devient suffisamment bien posé et le résultat voulu devrait être décrit avec la plus grande précision. Nous passons maintenant à l'étape de résolution la plus importante.

2.2. Deuxième étape : Spécification et analyse du problème

Cette étape consiste d'abord à spécifier la tâche du programme que nous comptons écrire. Ensuite, on procède à l'analyse du problème pour lui trouver une solution. Il existe plusieurs approches pour analyser un problème.

Nous en citons les plus connues à savoir l'approche ascendante et l'approche descendante. La première approche consiste à partir des données et commencer à développer les traitements nécessaires jusqu'à arriver au résultat escompté.

La seconde approche part du résultat du problème posé et au fur et à mesure, on définit chaque nouvel élément qui apparaît dans l'analyse. Pour trancher, il n'y a pas une approche meilleure que l'autre, chacune a ses avantages et ses inconvénients. D'ailleurs, on adopte souvent les deux approches selon les passages de l'analyse.

2.2.1. Exemple du calcul de la surface d'un rectangle

Problème :

Ecrire un programme qui calcule et affiche la surface d'un rectangle dont la longueur et la largeur sont données.

2.2.2. Solution par les deux approches

- **Approche ascendante**

Les données sont : Longueur et Largeur

Le traitement :

Calcul de la surface :

Surface = Longueur x Largeur

La sortie

Afficher Surface

Fin du programme

- **Approche descendante**

Résultat : Afficher (Surface)

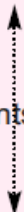
Surface = Longueur x Largeur

Longueur = DONNEE

Largeur = DONNEE

Fin du programme

Nous remarquons que l'approche descendante dicte un certain comportement systématique tout le long du développement de la solution. En effet, tout nouvel élément apparu dans l'analyse est placé à son tour à gauche pour être défini. De cette manière, on ne risque pas d'oubli d'un élément quelconque nécessaire à la résolution du problème en question. Notons que cette méthode impose à ce qu'on ait en tête une solution claire et précise au problème posé. Pour systématiser cette opération et en vue d'une organisation plus claire permettant un développement rapide et organisé de cette étape, nous allons utiliser la grille suivante :

NOM = nom_du_programme		
S (Séquences)	L.D.E. (Liste des Définitions Explicites)	O.U. (Objets Utilisés)
	Résultat =  Les éléments de l'analyse Fin nom_du_programme	

La première ligne de la grille comporte le nom que nous aurions choisi pour notre programme. La colonne centrale comporte explicitement toutes les définitions des objets constituant la solution du problème en suivant une analyse descendante comme indiquée dans l'exemple ci-dessus.

Au fur et à mesure que nous avançons dans l'analyse du problème, nous inscrivons les nouveaux objets dans la 3^{ème} colonne pour ne pas oublier de les définir.

La première colonne à gauche intitulée "Séquences" sert à ordonner les actions évoquées dans l'analyse par ordre chronologique d'exécution. Cette opération est très importante pour l'écriture de l'algorithme.

Reprenons l'exemple précédent du calcul de la surface d'un rectangle. L'analyse qui a été faite précédemment pourra être placée dans la grille comme suit :

NOM = Surface_ Rectangle		
S (Séquences)	L.D.E. (Liste des Définitions Explicites)	O.U. (Objets Utilisés)
4 3 1 2 5	Résultat = Afficher (Surface) Surface ← Longueur x Largeur Longueur = Donnée Largeur = Donnée Fin Surface_ Rectangle	Surface Longueur Largeur

La colonne des séquences comporte un ordre logique d'exécution des différentes actions qui ont constitué ce programme.

Remarque : un problème peut avoir plusieurs solutions.

2.3. Troisième étape : Écriture de l'algorithme

L'analyse d'un problème telle qu'elle est faite avec la grille devra permettre à la fin de dresser dans l'ordre croissant d'exécution les différentes instructions ou actions qui ont formé la solution du problème. L'écriture des instructions dans cet ordre croissant ou chronologique d'exécution donne un algorithme solution au problème posé. Toutefois, il est nécessaire de faire des conventions d'écriture et de se mettre d'accord sur un même langage pour que cette écriture soit comprise de la même façon par différentes personnes. Un certain vocabulaire muni d'une certaine syntaxe formeront ce qu'on appelle le **langage algorithmique**.

Rappelons qu'un algorithme est une suite ordonnée et finie d'actions ou d'instructions dont l'exécution servira à résoudre un problème donné.

Ecrivons l'algorithme déduit de l'analyse du problème relatif au calcul de la surface d'un rectangle :

```
0) Début SurfaceRectangle
1) Lire(Longueur)
2) Lire(Largeur)
3) Surface ← Longueur * Largeur
4) Ecrire(Surface)
5) Fin SurfaceRectangle
```

Nous attirons votre attention sur les conventions suivantes :

- l'utilisation du verbe « Lire » pour la saisie d'une donnée et du verbe « Ecrire » pour l'affichage ou la sortie du résultat.
- l'utilisation du signe « ← » pour traduire l'affectation d'une variable par une valeur.

Ce vocabulaire et sa syntaxe d'utilisation seront développés en détail dans les prochains chapitres.

Exercices

1. On se propose d'écrire un programme qui permet de calculer et d'afficher à l'écran, la moyenne annuelle Moy d'un élève. On donne la formule de calcul suivante : $Moy = (Note_trim1 + 2 * Note_trim2 + 2 * Note_trim3)/5$
2. Ecrire un algorithme qui permet de calculer et d'afficher pour une voiture, la consommation de carburant en litres par 100 km sachant qu'elle a consommé x litres d'essence pour parcourir d km. d et x sont des données.

2.4. Quatrième étape : Ecriture du programme

Une fois l'algorithme solution d'un problème est établi ou écrit, l'étape suivante sera de penser à la machine et comment le traduire pour qu'il soit compris et exécuté par cette dernière. Les langages avec lesquels nous allons traduire cet algorithme sont appelés *langages de programmation*. Ils sont dits évolués relativement au langage binaire et au langage assembleur qui sont plus proches de la machine. Il existe plusieurs langages évolués. Ces derniers ont été souvent développés selon des

besoins spécifiques. Avec l'évolution de l'informatique et en vue d'étendre l'activité de programmation au plus large public, plusieurs langages ont vu le jour.

Un langage de programmation sert à traduire un algorithme en un programme source compréhensible directement ou indirectement par l'ordinateur pour que ce dernier puisse l'exécuter.

Il est composé comme tout langage, d'un vocabulaire obéissant à une sémantique et a une syntaxe. La sémantique donne un sens à chaque mot de ce langage par contre la syntaxe détermine les règles d'utilisation.

Ces langages sont très stricts du côté vocabulaire et syntaxe. Ils sont souvent munis d'un analyseur permettant de détecter les erreurs faites sur le vocabulaire et la syntaxe.

Ces langages sont souvent munis d'un éditeur de texte permettant au programmeur de saisir son programme. Le fichier obtenu est appelé programme source. Ce dernier est compréhensible par un lecteur qui connaît bien le langage.

L'exécution de ce programme dépend du langage utilisé. Si le langage est **interprété**, le programme source sera exécuté instruction par instruction. Par contre, si le langage est **compilé**, il faudra faire la compilation du programme source et corriger les éventuelles erreurs pour produire un autre programme dit programme exécutable. C'est ce dernier, écrit en langage machine par le langage lui-même, qui, sera exécuté. Les éventuelles erreurs qui se produiront seront dites erreurs d'exécution.

Un langage compilé donne des programmes exécutables dont l'exécution est très rapide relativement à l'exécution d'un programme interprété.

En voici un tableau des langages de programmation les plus connus.

Nom	Origine	Année	Caractéristiques
FORTRAN : (Formula translation)	J. BACKUS, IBM	1954-1957	Compilé, Applications scientifiques (mathématiques).
LISP : LISt Processing	J. BACKUS, IBM	1956-1961	Langage fonctionnel (interprété ou compilé). Applications pour intelligence artificielle.
Algol : (Algorithmic Language)	J. Backus, P. Naur	1958-1960	Compilé, Langage à structure de blocs.
APL : (A Programming Langage)	IBM	1960	Interprété, Applications mathématiques .
COBOL : (Commun Business Oriented Language)	US DOD GPO et CODASYL Committee	1960	Compilé, Applications de gestion et de commerce.

Nom	Origine	Année	Caractéristiques
BASIC : (The Beginners All-purpose Symbolic Instruction Code)	J. KEMENY, T. KURTZ (à Darmouth)	1963-1964	Interprété, Syntaxe simple, larges applica- tions. Très répandu avec les micros.
PASCAL : (hommage au savant Blaise PASCAL)	Niklaus WIRTH	1968-1971	Compilé, Applications générales.
C	Richie et Thomson	1972-1973	Compilé, Langage à structuration de blocs
PROLOG : (PROgramming in LOGic)	D.H. WARREN et R.KOWALSKI	1979	Souvent interprété, Applications mathématiques et logiques.
C++	AT&T Bell Labs	1982-1985	Compilé, Langage orienté objets.
PERL : Practical Extraction and Reporting Language	Larry WALL	1986	Interprétation de script. Applications générales.
JAVA	SUN Microsystems	1994-1995	Langage orienté objet. Applications diverses.

2.5. Cinquième étape : Exécution et tests du programme

Bien entendu, un programme est écrit pour être exécuté et résoudre un problème donné. Souvent et à cause d'une erreur quelconque, le programme s'exécute normalement mais ne répond pas favorablement à certains cas. Il est donc impératif de faire subir au programme obtenu, une série de tests bien choisis pour s'assurer que ce dernier fait bien ce qu'on lui demande de faire dans tous les cas possibles.

Exercices

1. Formulez au moins deux problèmes à résoudre avec l'ordinateur dans chacun des domaines suivants :
 - Les mathématiques
 - La physique
 - Les langues
2. Donnez les différentes étapes à suivre pour résoudre un problème, depuis la position du problème jusqu'aux tests du programme.
3. Faites une analyse et en déduire un algorithme qui conjugue un verbe du 1er groupe au passé composé. (Définir le résultat puis chaque constituant de ce résultat).
4. Ecrivez un algorithme qui saisit un verbe du 1er groupe et le conjugue à l'imparfait. (Définissez le résultat puis chaque constituant de ce résultat).
5. On se propose de résoudre une équation du premier degré dans l'ensemble des réels de la forme $a.x + b = 0$. Effectuez une analyse descendante de ce problème et en déduisez l'algorithme correspondant.
6. On veut déterminer le plus grand de deux réels données a et b . Trouvez une expression pour traduire ce résultat. Trouvez une formule mathématique donnant directement ce résultat.
7. Mêmes questions que l'exercice 6 pour la détermination du plus petit élément de deux réels données a et b .
8. Donnez au moins deux caractéristiques d'un langage de programmation.
9. Citez deux langages interprétés et deux autres compilés.
10. Quelles sont les différences entre un langage interprété et un langage compile.

Chapitre 3 : Les structures de données et les structures algorithmiques simples

3.1. Introduction au troisième chapitre

Le travail d'un programme s'articule autour de la manipulation de valeurs contenues dans des objets qui peuvent être des constantes ou des variables. Ce chapitre contient un rappel de ces notions en plus des types de données les plus utilisés. Nous aborderons par la suite de nouvelles structures qui sont nécessaires pour la résolution de certains problèmes.

3.2. Constantes et variables

3.2.1. Constantes

Une constante est un objet dont la valeur reste fixe durant l'exécution d'un programme.

- **Caractéristiques**

Une constante est caractérisée par un nom qui est son identificateur unique en plus par une valeur inchangeable qui nous permet de déterminer implicitement le type de cette dernière.

- **Déclaration algorithmique**

En algorithmique, on déclare une constante comme suit :

	Objet	Type / Nature	Rôle
Général	Nom	Constante = Valeur de la constante	Rôle
Exemple	P1	Constance = 9.8	

Exercice

On se propose de calculer l'allongement L d'un ressort de raideur K auquel est accrochée une masse m . Sachant que : $m * g = K * L$ et $g = 9.8$.

- Présentez la spécification de ce problème.
- Déduisez l'algorithme correspondant.

3.2.2. Variables

Une variable est un emplacement de la mémoire que l'on réserve pour stocker des informations de même type. Elle est désignée par un nom appelé *identificateur*.

- **Caractéristiques**

Une variable est caractérisée par :

- Un nom (identificateur unique)
- Un contenu
- Un type

Remarque :

Il est préférable que le nom donne à une variable soit évocateur de l'information qu'il désigne. Cependant certaines règles doivent être respectées lors du choix de cet identificateur, à savoir :

- Il doit commencer par une lettre ;
- Il ne doit pas contenir les espaces, ni les caractères accentués ;
- Il ne doit pas utiliser les mots réservés ;
- Les noms sont composés en utilisant le caractère de soulignement (`_`).
Exemple : `Nom_Etudiant`

Le type d'une variable est l'ensemble de valeurs que peut prendre cette variable.

- **Types de variables**

Nous distinguons trois types principaux de variables :

a. Les variables chaînes

Appelées aussi variables alphanumériques, elles représentent les données dont le contenu est constitué des lettres, chiffres et des caractères spéciaux. La taille maximale d'une variable chaîne est de 255 caractères.

b. Les variables numériques

Elles représentent les données dont le contenu est constitué des nombres. De ce fait, on y distingue celles qui sont des entiers (Entiers courts et Entiers longs) et celles qui sont des décimaux simples et doubles (Réels).

Bref, les types numériques sont :

- Entiers courts
- Entiers longs
- Réels

c. Les variables booléennes

Ce sont des variables qui ne prennent que deux valeurs mutuellement exclusives : Vrai ou Faux.

3.3. Opérateurs

3.3.1. Opérateurs Arithmétiques

Multiplication	: *
Division	: /
Division entière	: \ ou DIV
Division modulaire	: MOD
Addition	: +
Soustraction	: -
Exponentiation	: ^ ou **
Carré	: SQR()
Racine carrée	: SQRT()

3.3.2. Opérateurs relationnels

Opérateur	Fonction
=	Egal
<	Inférieur
>	Supérieur
<=	Inférieur ou égal
>=	Supérieur ou égal
<>	Différent

3.3.3. Opérateurs logiques

Ils permettent de réunir deux opérations conditionnelles.

- AND (Et) : P AND Q est vrai si P et Q sont à tous à la fois vrais.
- OR (Ou) : P OR Q est faux si P et Q sont tous à la fois faux.
- NOT (Non) : Cet opérateur est unaire. C'est un inverseur de valeur. NOT(P) est vrai si P est faux et il est faux si P est vrai.

3.3.4. Fonctions standards

Nous présentons ici quelques fonctions standards en algorithmique et leurs équivalents en langage PASCAL dont les détails seront donnés dans le chapitre suivant.

Algorithmique	Pascal	Rôle	Exemples
Trunc (x)	Trunc (x)	Permet d'extraire la partie entière de x.	Trunc (5, 2) vaut 5 Trunc (5, 9) vaut 5
Arrondi (x)	ROUND (x)	Arrondit une valeur réelle à l'entier le plus proche.	ROUND (10.23) vaut 10 ROUND (10.5) vaut 11 ROUND (10.83) vaut 11
ABS (x)	ABS (x)	Renvoie la valeur absolue de x.	ABS (-10) vaut 10
Carré (x)	SQR (x)	Renvoie le carré de x.	SQR (6) vaut 36
Racine Carré (x)	SQRT (x)	Renvoie la racine carrée de x s'il est positif sinon elle provoque une erreur.	SQRT(5) vaut 2.236
INT (x)	INT (x)	Renvoie la partie entière de x	INT(10.23) vaut 10.00
FRAC (x)	FRAC (x)	Renvoie la partie décimale de x	FRAC (10.23) vaut 0.23
Cos (x)	Cos (x)	Renvoie le cosinus de x (x en radians)	COS (PI) vaut -1.00
Sin (x)	Sin (x)	Renvoie le sinus de x (x en radians)	SIN (PI) vaut 0.00

Exercice

- Ecrivez les formules suivantes en langage algorithmique :
 - $f(x) = x^4 - 3x^2 + 1$
 - $g(x) = x^3 - \sin(x)$
 - $h(x) = E(x) - 5$
- Recopiez puis ajoutez au tableau précédent deux colonnes :
 - Une première colonne pour exprimer le type du paramètre x.
 - Une seconde pour le type du résultat
- Sachant que $a = 4$, $b = 5$, $c = 1$ et $d = 0$, évaluez les expressions logiques suivantes :
 - $(a < b)$ et $(c \geq d)$
 - Non $(a < b)$ ou $(c \neq b)$
 - Non $(a \neq b^2)$ ou $(a * c < d)$

Remarque :

Lors de l'évaluation d'une expression, il faut tenir compte des règles de priorité entre les opérateurs utilisés.

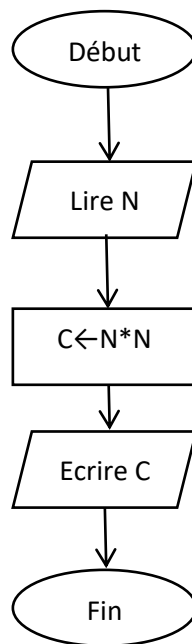
3.4. Structures algorithmiques simples

Une structure est dite simple (appelée encore une séquence), si elle ne contient que des instructions :

- d'entrée de données,
- d'affectation,
- de sortie de résultats.

La séquence est constituée d'une suite finie d'instructions qui s'exécutent linéairement et inconditionnellement. Les traitements sont effectués les uns à la suite des autres.

Exemple



Remarques

- la source de lecture par défaut est le clavier, c'est le moyen le plus utilisé, mais elle peut être aussi une mémoire auxiliaire, ...
- La destination d'écriture est par défaut l'écran, elle peut être aussi une imprimante, un support de stockage, ...

3.4.1. Opération d'entrée

Une entrée est une opération de lecture de données. C'est une opération qui permet d'affecter, à une variable, une donnée introduite à partir d'un périphérique d'entrée tel que le clavier. Les valeurs lues seront copiées dans des variables en mémoire centrale de l'ordinateur.

a. Déclaration

Syntaxe en algorithmique	Syntaxe en Pascal
Lire (variable)	Read (variable) ; ou bien Readln (variable) ;

b. Exemples

Algorithmique	Pascal
Lire (x) Lire (a,b,c)	Read (x) ; ou bien Readln (x); Read (a,b,c) ; ou bien Readln (a,b,c);

Remarques

- Il doit y avoir correspondance de type (ou compatibilité) entre les données qui sont lues et les variables réservées en mémoire et qui vont recevoir les valeurs lues.
- L'instruction Readln force le passage à la ligne suivante après la lecture des différentes variables figurant dans l'instruction.

3.4.2. Opération de sortie

Une sortie est une opération d'écriture de données (souvent des résultats de traitements). C'est une opération qui permet d'écrire, sur un périphérique de sortie, les contenus de variables en mémoire centrale.

a. Déclaration

	Syntaxe en algorithmique	Syntaxe en Pascal
Affichage d'un texte	Ecrire ("texte")	Write ('texte');
Affichage du contenu d'une variable	Ecrire (variable)	Write (variable) ; ou Writeln (variable) ;
Affichage d'un texte + contenu d'une variable	Ecrire ("texte", variable)	Write ('texte', variable) ; ou Writeln ('texte', variable) ;

b. Exemples

Algorithmique	Pascal
Ecrire (x)	Write (x) ;
Ecrire (a,b,c)	Write (a,b,c) ;
Ecrire("La valeur de a est ", a)	Write('La valeur de a est', a);
Ecrire("L'élève ", nom, " a une moyenne", moy, " / 20")	Write('L'élève', nom, ' a une moyenne', moy, ' / 20') ;

Remarques

- En Pascal les commentaires sont placés entre apostrophes.
- L'instruction WriteLn permet de passer à la ligne suivante après l'écriture des différentes variables et textes figurant dans l'instruction.

3.4.3. Opération d'affectation

L'opération d'affectation permet d'affecter une valeur à une variable. Elle est représentée par une flèche orientée vers la gauche « $\leftarrow$ ».

a. Vocabulaire et syntaxe

Syntaxe en algorithmique	Syntaxe en Pascal
Variable $\leftarrow$ valeur	Variable := valeur ;

b. Exemples

Algorithmique	Pascal	Commentaires	Résultats
$x \leftarrow 5$	$x := 5 ;$	x reçoit 5	$x = 5$
$a \leftarrow x + 3$	$a := x + 3 ;$	a reçoit la valeur de l'expression $x + 3$	$a = 5 + 3 = 8$
$a \leftarrow x - 2$	$x := x - 2 ;$	x reçoit la valeur de l'expression $x - 2$	$x = 5 - 2 = 3$

Remarques

- Dans la première affectation la variable x prend la valeur 5. Quel que soit le contenu de x, il sera remplacé par 5.
- La variable intitulée a, aura pour valeur la somme de 5 et 3 donc 8.
- La variable x aura pour valeur la différence entre la valeur actuelle de x (qui est 5) et 2. Donc la nouvelle valeur de x est 3 ($3 = 5 - 2$)

Exercices

1. Soit la séquence d'instructions suivante :

1. $A \leftarrow 5$
2. $B \leftarrow 7$
3. $A \leftarrow A + B$
4. $B \leftarrow A - B$
5. $A \leftarrow A - B$

- Dressez un tableau pour déterminer les valeurs des variables A et B après l'exécution des instructions précédentes.
- Quel est le rôle des instructions 3,4 et 5 ?

Solution :

N° instruction	A	B
1	5	
2	5	7
3	12	7
4	12	5
5	7	5

Cette séquence d'instructions permet de permuter les contenus des deux variables A et B.

2. Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « Ap1_V1.0 ».

Solution :

```

PROGRAM AP1_V1.0 (INPUT, OUTPUT) ;
USES CRT ;
VAR
A, B : REAL ;
BEGIN
A := 5 ;
B := 7 ;
A := A + B ;
B := A - B ;
A := A - B ;
END.

```

3. On se propose maintenant de trouver une autre méthode pour réaliser une permutation.

Spécification du problème

Résultat : Ecrire(A, B)

Traitement : Pour pouvoir permuter les valeurs des deux variables A et B, nous pouvons faire recours à une variable intermédiaire que nous allons appeler C. Les actions à faire dans l'ordre sont :

```

C ← A
A ← B
B ← C

```

A, B = données

Algorithme :

```

DEBUT Algorithme Permutation
// cet algorithme permet de permuter les valeurs de deux variables A et B //
Variables :
A, B : Entiers
Début action principale
Ecrire("A = "), Lire(A)
Ecrire("B = "), Lire(B)
C ← A
A ← B
B ← C
Ecrire( A, B )
Fin action principale
FIN Permutation

```

Codification des objets :

Nom	Type	Rôle
A	Entier	Une variable contenant la première valeur
B	Entier	Une variable contenant la deuxième valeur
C	Entier	Une variable intermédiaire

4. Traduire l'algorithme de la question numéro 3 en PASCAL

Solution

```

PROGRAM Permutation (INPUT, OUTPUT) ;
USES CRT ;
VAR
A, B, C : INTEGER ;
BEGIN
WRITE('A =') ;
READLN(A) ;
WRITE('B =') ;
READLN(B) ;
C := A ;
A := B ;
B := C ;
WRITELN('A =', A) ;
WRITELN('B =', B) ;
END.

```

5. Calculer et afficher le quotient et le reste de la division Euclidienne de A par B.

Solution :

Spécification du problème

Résultat : Ecrire (q, r)

Traitement : Pour avoir le quotient (q) et le reste (r) de la division Euclidienne (ou entière) de A par B, il suffit d'utiliser les fonctions prédéfinies DIV et MOD. En effet :

$q \leftarrow A \text{ DIV } B$

$r \leftarrow A \text{ MOD } B$

A, B = données

Algorithme :

DEBUT Algorithme Euclide

// cet algorithme permet de calculer puis d'afficher le quotient et le reste de la division Euclidienne de deux variables A par B //

Variables :

A, B, q, r : Entiers

Début action principale

Ecrire("A = "), Lire(A)

Ecrire("B = "), Lire(B)

q A DIV B

r A MOD B

Ecrire("Le quotient est ",q, " et le reste est ",r)

Fin action principale

FIN Euclide

Codification des objets :

Nom	Type	Rôle
A	Entier	Une variable contenant la première valeur
B	Entier	Une variable contenant la deuxième valeur
q	Entier	Une variable qui va contenir la valeur du quotient
r	Entier	Une variable qui va contenir la valeur du reste

6. Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « **Ap2_V1.0** ».

7. Utilisez le même principe pour convertir en heures, minutes et secondes un temps donne en secondes.

8. Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « **Ap3** ».

3.5. Types énumérés

Les types énumérés permettent de représenter des valeurs en les énumérant au moyen de leurs noms. En effet, un type énuméré est un type dont les variables associées n'auront qu'un nombre limité de valeurs. En Pascal, cette valeur est d'au maximum 256.

La définition d'un type énuméré consiste à déclarer une liste de valeurs possibles associées à un type, c'est-à-dire qu'une variable de type énuméré aura l'une et une seule de ces valeurs et pas une autre.

3.5.1. Exemple

Semaine = (Dimanche, Lundi, Mardi, Mercredi, Jeudi, Vendredi, Samedi)

Type_couleur = (Rouge, Bleu, Vert, Jaune, Blanc, Noir)

Semaine	Dimanche	Lundi	Mardi	Mercredi	Jeudi	Vendredi	Samedi
Type_couleur	Rouge	Bleu	Vert	Jaune	Blanc	Noir	

Remarque :

La partie déclarative de cet exemple

Semaine = (Dimanche, Lundi, Mardi, Mercredi, Jeudi, Vendredi, Samedi) aurait bien pu être raccourcie en :

Jour : (Dimanche, Lundi, Mardi, Mercredi, Jeudi, Vendredi, Samedi)

3.5.2. Déclaration

- *Au niveau de l'algorithme*

Objet	Type / Nature	Rôle
Nom	Liste de valeurs	Rôle

- *Au niveau de Pascal*

Type Nom_type = (valeur1, valeur2, valeur3, ...) ;

Var Nom : Nom_type ;

Ou bien

Var Nom : (valeur1, valeur2, valeur3, ...) ;

Exercice

Donnez les déclarations des types énumérés suivants :

- Matière comportant les noms de cinq matières de votre choix
- Sens comportant les quatre sens de direction qui sont haut, bas, gauche et droite
- Booléen pouvant être vrai ou faux

3.5.3. Fonctions prédéfinies sur les types énumérés

Avant de présenter les fonctions et les procédures susceptibles d'être utilisées avec les types énumérés, il est utile de donner les deux constations suivantes :

Avant de présenter les fonctions et les procédures susceptibles d'être utilisées avec les types énumérés, il est utile de donner les deux constatations suivantes :

- a) Les valeurs d'un type énuméré sont ordonnées selon leur ordre de déclaration. Pour le type Semaine, nous avons : Dimanche < Lundi < Mardi < Mercredi < ...
- b) Les types énumérés font partie des types scalaires, en effet à chaque valeur énumérée correspond un numéro d'ordre (nombre entier). La première valeur porte le numéro 0, la seconde le numéro 1, etc.

Dans notre exemple, le type semaine est un type énuméré composé de 7 éléments représentant les jours de la semaine. Remarquez que les éléments sont uniquement des identifiants qui n'ont aucune valeur intrinsèque. On peut juste les repérer par leur index (l'ordre dans lequel ils apparaissent dans la déclaration. Dimanche porte le numéro 0, Lundi porte le numéro 1, et Samedi porte le numéro 6.

1) Les opérations possibles (en plus de l'affectation et du passage en paramètre) sur les valeurs d'un type énuméré sont : =, ≠, <, >, ≤, >

2)

Fonction	Rôle	Exemples
Succ (exp_type_énuméré)	Fournit le successeur de la valeur donnée par exp_type_énuméré (erreur si cette expression désigne la dernière valeur énumérée)	Succ(Lundi) donne Mardi
Pred (exp_type_énuméré)	Fournit le prédécesseur de la valeur donnée par exp_type_énuméré (erreur si cette expression désigne la première valeur énumérée)	Pred(Lundi) donne Dimanche
Ord (exp_type_énuméré)	Fournit le numéro d'ordre de la valeur donnée par exp_type_énuméré	Ord(Dimanche) donne 0 Ord(Jeudi) donne 4

3.6. Types utilisateurs

3.6.1. Déclaration

- *Au niveau de l'algorithme*

Tableau de déclaration de nouveaux types (T.D.N.T)

Type
Nom type = (val1, val2,..., valn)

Objet	Type/Nature	Rôle
Nom variable	Nom type	Rôle

- *Au niveau de Pascal*

Type Nom type = (val1,val2,...,valn);

Var Nom variable : nom type;

3.6.2. Remarque

Si l'ensemble de valeurs que peut prendre ce nouveau type forme un intervalle, on peut les représenter sous la forme générale suivante Binf ..Bsup

Exercices

Déclarez les types suivants :

- Un type intitulé mention qui n'acceptera que les mentions attribuées à une moyenne d'un élève (Passable, assez bien, bien, très bien)
- Un type saison comportant les quatre saisons (Automne, Hiver, Printemps, Eté).

3.7. Tableaux

Un tableau est un regroupement de valeurs portant le même nom de variable et repérées par un numéro. Il permet de ranger un nombre fini d'éléments de même type et selon une disposition bien définie. Le numéro qui permet de repérer chaque valeur s'appelle l'indice. Chaque fois que l'on veut désigner un élément du tableau, on fait figurer le nom du tableau, suivi de l'indice de l'élément, entre crochets.

3.7.1. Déclaration

Première formulation

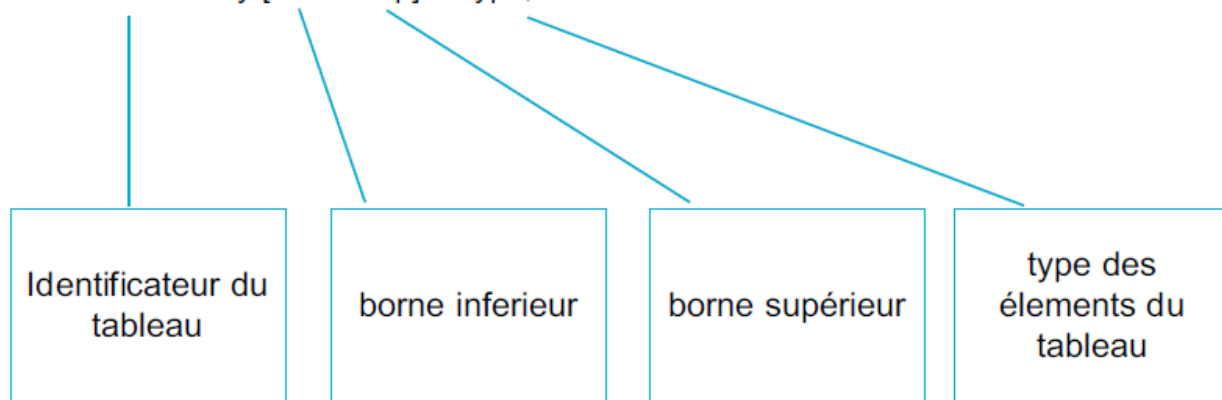
- *Au niveau de l'algorithme*

Objet	Type/Nature	Rôle
Nom tableau	Tableau de taille de type	Rôle

- *Au niveau de Pascal*

Var

Nom : Array [Binf..Bsup] of type;



Exemple

var Moyenne : Array [1..10] of real;

Deuxième formulation

- *Au niveau de l'algorithme*

Tableau de déclaration de nouveaux types

Type
Nom type = tableau de taille et de type

T. D. O

Objet	Type/Nature	Rôle
Nom tableau	Nom type	Rôle

- *Au niveau de Pascal*

Type Nom_type = Array [Binf..Bsup] of type ;

Var Nom tableau : Nom_type ;

Exercices

Ecrivez un programme qui permet la saisie de 6 valeurs puis d'afficher leurs carrés
Le dialogue avec l'utilisateur se présente comme suit : Donnez 6 nombres entiers :
12, 4, 8, 58, - 9, 11

Nombre	Carré
12	144
4	16
8	64
58	3364
-9	81
11	121

3.7.2. Tableaux à deux dimensions

Première formulation

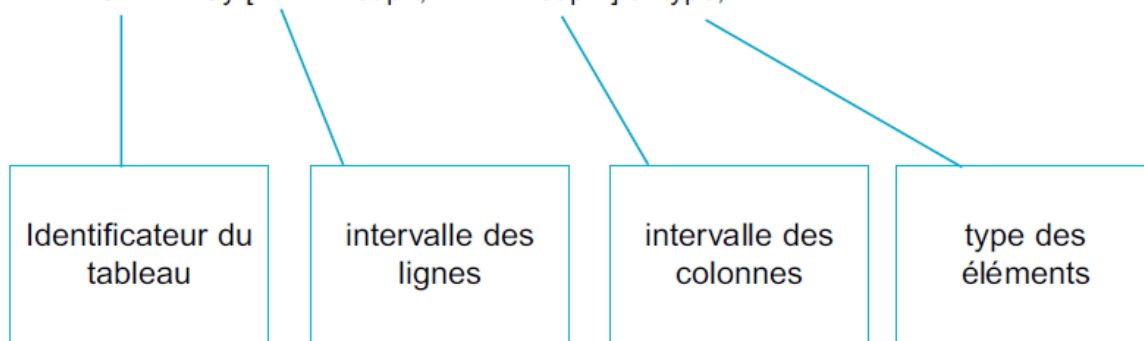
- *Au niveau de l'algorithme*

Objet	Type/Nature	Rôle
Nom tableau	Tableau de taille de type	Rôle

- *Au niveau de Pascal*

Var

Nom : Array [Binf1..Bsup1, Binf2..Bsup2] of type;



Deuxième formulation

- *Au niveau de l'algorithme*

Tableau de déclaration de nouveaux types

Type
Nom type = tableau de taille et de type

Tableau de déclaration des objets

Objet	Type/Nature	Rôle
Nom tableau	Nom type	Rôle

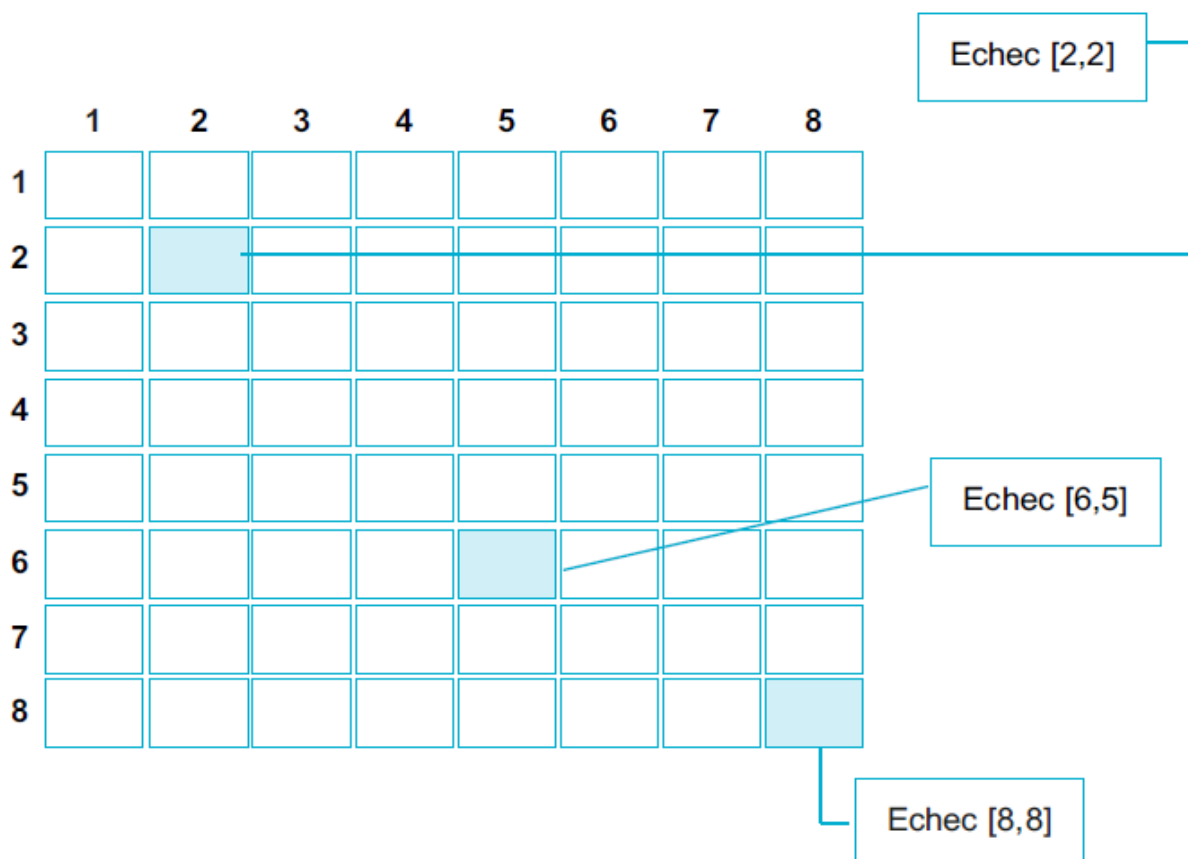
- *Au niveau de Pascal*

Type Nom_type = Array [Binf1..Bsup1, Binf2..Bsup2] of type ;

Var Nom tableau : Nom_type;

Exemple

Echec : Array [1..8, 1..8] of integer;



Echec[2,2] correspond à l'élément de la ligne n°2 et de la colonne n°2 autrement dit, il s'agit du 2^{ème} élément de la 2^{ème} ligne. Echec[5,6] correspond au 6^{ème} élément de la ligne n°5.

Echec[8,8] correspond à l'élément n°8 de la colonne n°8 Pour accéder à un élément du tableau Echec, il suffit de donner l'identificateur du tableau (ou son nom) et les deux indices de cet élément (son numéro de ligne et son numéro de colonne).

Exercice

On se propose d'écrire un programme qui permet de calculer la moyenne de n élèves relativement aux notes de 5 matières. Sachant que le coefficient de chaque matière est de 1.

- a) Ecrivez une spécification pour ce problème.
- b) Déduisez l'algorithme correspondant.
- c) Traduisez cet algorithme en Pascal.

Chapitre 4 : Les structures algorithmiques de contrôle

Ce chapitre présente les structures algorithmiques de base. Les applications présenteront les nouvelles structures de données traitées dans le chapitre précédent.

4.1. Structures de contrôle conditionnelles

Les structures de contrôle conditionnelles permettent à un programme de modifier son traitement en fonction d'une ou de plusieurs conditions.

Ces structures sont de trois types :

- la structure conditionnelle simple
- la structure conditionnelle généralisée
- la structure a choix.

Dans la suite du cours, nous allons rappeler et appliquer ces structures dans la résolution de quelques problèmes.

4.1.1. Structure conditionnelle simple

Une condition est une expression booléenne qui peut prendre la valeur vraie ou la valeur fausse.

La structure conditionnelle simple peut être représentée comme suit :

Syntaxe en algorithmique	Syntaxe en Pascal
Si conditions alors	If conditions Then
	Begin
Traitement1	Traitement1;
Sinon	End
	Else
Traitement2	Begin
	Traitement2;
Fin si	End;

Exemple :

Si age > 18 **Alors**

Etat ← 'Majeur'

Sinon

Etat ← 'Mineur'

Fin Si

a. **Si** condition **Alors**

Actions 1

Sinon Actions 2

Une Alternative utilise trois éléments :

- La condition introduite par Si
- La clause alors
- La clause sinon
- Fin Si pour certaines structures

b. **Si** Condition **Alors**

Bloc d'actions 1

Sinon

Bloc d'actions 2

Fin Si

Exemple didactique

Résoudre un problème qui demande un nombre à l'utilisateur et l'informe ensuite si ce nombre est pair ou impair.

Spécification du problème

Résultat : Ecrire (nature)

Traitement : La détermination de la nature du nombre saisi (qu'on appellera x) se fait en testant le reste de la division (r) de ce nombre par 2. En effet, s'il est égal à 0, la valeur de la variable nature est « Paire », sinon la valeur de la variable nature est « Impaire ».

Si $r = 0$ alors

Nature $\leftarrow$ " Paire "

sinon

Nature $\leftarrow$ " Impaire "

Fin si

$r \leftarrow x \text{ MOD } 2$

$x = \text{donnée}$

Algorithme :

DEBUT Algorithme nombre

// cet algorithme permet d'afficher la nature d'un nombre saisi //

Variables

x, r : Entiers

Nature : Chaîne

Debut action principale

Ecrire("x = "), Lire(x)

$r \leftarrow x \text{ MOD } 2$

Si ($r = 0$) alors

Nature $\leftarrow$ "Paire"

Sinon

Nature $\leftarrow$ "Impaire"

Fin si

Ecrire(x , " est ", Nature)

Fin action principale

FIN Nombre

Codification des objets :

Nom	Type	Rôle
x	Entier	Une variable contenant un nombre.
r	Entier	Une variable qui recevra le reste de la division de x par 2.
Nature	Chaîne de caractères	Une variable contenant la nature de x (pair ou impair).

T.D. Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « **Act2** ».

La structure conditionnelle simple est une structure algorithmique qui fait appel à au maximum deux traitements suivant le résultat de l'évaluation de la condition.



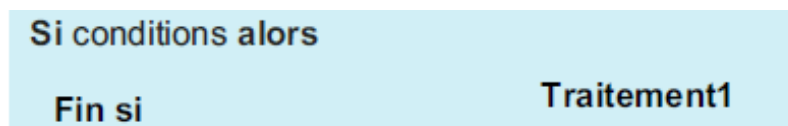
Remarquons, tout d'abord, que traitement1 et traitement2 peuvent être forme chacun d'une seule ou de plusieurs instructions.

L'exécution d'une telle structure commence par l'évaluation de la condition qui donne un résultat de type booléen. Si ce booléen a pour valeur **VRAI**, alors **Traitement_1** sera exécuté, dans le cas contraire c'est **Traitement_2** qui sera exécuté.



Remarques :

1. La condition peut être composée.
2. Si Traitement2 est vide, on parle de structure conditionnelle simple réduite qui a la syntaxe suivante :



Exercices

1. Exécutez manuellement l'algorithme suivant et déduisez les valeurs finales des variables a, b et c pour chacune des valeurs initiales suivantes de a :

i) $a = 10$

ii) $a = 3$

iii) $a = 0$

DEBUT Algorithme Application

Variables

a, b, c : Réels

Début action principale

Ecrire("a = ") Lire(a)

$b \leftarrow 2 * a$

$c \leftarrow b - 4$

Si $a > 2 * c$ alors

$b \leftarrow a - 2 * c$

Sinon

$b \leftarrow a + c$

Fin si

Ecrire(a,b,c)

Fin action principale

FIN Application

Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « **Ap1** ».

2. Lire un nombre et l'afficher s'il est supérieur à 100.
3. Lire un nombre et l'afficher s'il est supérieur à 100 sinon afficher son double.
4. Lire un montant et accorder une prime de 10% si le montant est inférieur à 2000 sinon accorder une prime de 15%. Afficher la prime calculée.
5. On veut accorder une réduction à tous les clients selon les critères suivants :
 - Pour les clients ayant acheté les produits de la catégorie A : pour un montant de plus de 10.000 FC, une réduction de 10% sinon une réduction de 5%.
 - Pour les autres catégories, pour un montant de plus de 5000 FC une réduction de 3% sinon une réduction de 2%.

Présenter l'algorithme et l'ordinogramme.

4.1.2. Structure conditionnelle généralisée

La structure conditionnelle généralisée est une structure algorithmique qui fait appel à plus que deux traitements. L'exécution d'un traitement entraîne automatiquement la non-exécution des autres traitements.



Comme dans le cas d'une structure conditionnelle simple, chaque traitement peut comporter une ou plusieurs instructions.

L'exécution d'une telle structure commence par l'évaluation de **condition_1** qui donne un résultat de type booléen. Si ce booléen a pour valeur **VRAI**, alors **traitement_1** sera exécuté et l'exécution continue avec l'instruction qui suit la structure conditionnelle. Dans le cas contraire c'est **condition_2** qui sera évaluée, si elle est vérifiée alors **traitement_2** sera exécuté et on quitte la condition, sinon l'exécution continue de la même façon avec les autres conditions.

Si aucune condition n'est vérifiée alors c'est **traitement_(n+1)** qui sera exécuté.

Exercices

1. Résoudre un problème qui demande l'âge d'un enfant, puis l'informe de sa catégorie :

- "Papillon " moins que 6 ans
- " Poussin " de 6 à 7 ans
- " Pupille " de 8 à 9 ans
- " Minime " de 10 à 11 ans
- " Cadet " de 12 à 14 ans
- " Junior " de 15 à 17 ans
- "Senior " de 18 à 20 ans
- "Tu as dépassé l'âge de l'enfance " plus que 20 ans

2. Lire une température de l'eau comme une variable de type entier, puis afficher son état. Les trois formes de l'eau sont :

- " gaz " si la température dépasse 100°
- " liquide " si la température est entre 0° et 100°
- " glace " si la température est inférieure à 0°.

3. Ecrivez un programme qui saisit successivement un réel r1, un opérateur op et un deuxième réel r2 puis affiche le résultat de l'application de l'opérateur op sur r1 et r2.

4. Ecrivez un programme qui permet de lire une lettre (supposée majuscule) et afficher "Groupe A" si la lettre est "A", "Groupe B" si la lettre est "B" ; dans tous les autres cas afficher le message "Erreur de saisie".

4.1.3. Structure conditionnelle à choix

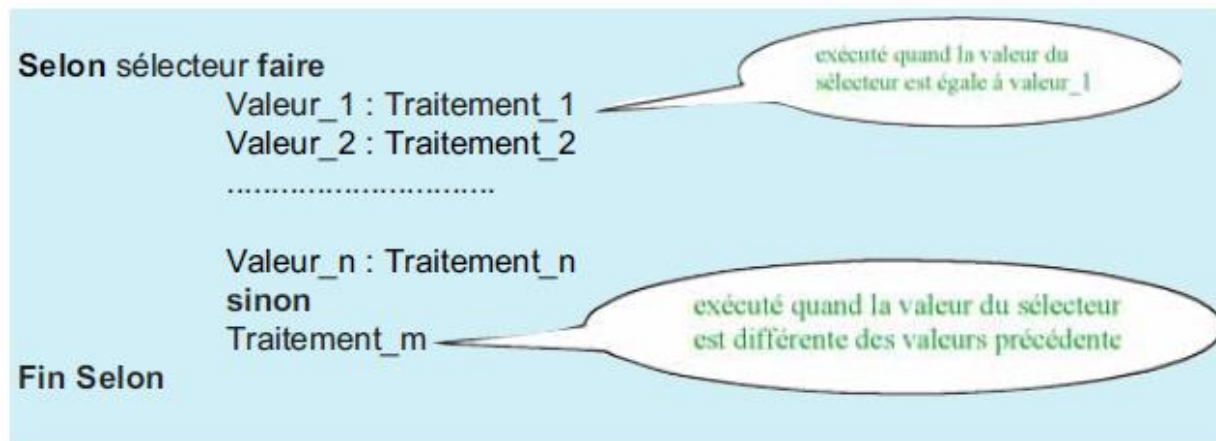
La structure conditionnelle à choix est une structure algorithmique utilisée de préférence dans des structures à plusieurs traitements selon des conditions. Comme le cas d'une structure conditionnelle généralisée, l'exécution d'un traitement entraîne automatiquement la non exécution des autres. Le choix d'un traitement se fait suivant la valeur d'un selecteur. Cette structure permet d'éviter le recours à une longue structure conditionnelle généralisée et offre une meilleure lisibilité de la solution.

```
Selon sélecteur faire
    Valeur_1 : Traitement_1
    Valeur_2 : Traitement_2
    Valeur_3 : Traitement_3
    .
    .
    Valeur_n : Traitement_n
sinon
    Traitement_m
Fin Selon
```

Remarques

- 1) Le sélecteur doit être de type scalaire discret et non réel.
- 2) Comme dans le cas des autres structures conditionnelles, chaque traitement peut comporter une ou plusieurs instructions.

- 3) Valeur_i peut être un ensemble de valeurs qui seront séparées par des virgules ou un intervalle de valeurs (vi .. vf)
- 4) Si la valeur du sélecteur est différente des valeurs proposées alors c'est le traitement qui suit la clause sinon qui sera exécuté.
- 5) La clause sinon et son traitement sont facultatifs dans la structure.



Exercice

1. Reprenez l'exercice 1 de la page 50, et cherchez une solution en utilisant la structure conditionnelle à choix.

L'application consiste à demander l'âge d'un enfant, puis cherche sa catégorie :

- " Poussin " de 6 à 7 ans
- " Pupille " de 8 à 9 ans
- " Minime " de 10 à 11 ans
- " Cadet " de 12 à 14 ans
- " Junior " de 15 à 17 ans
- " Espoir " de 18 à 20 ans
- " tu as dépassé l'âge de l'enfance" plus que 20 ans.

2. Ecrivez une analyse puis un programme d'un problème qui saisit une date en introduisant successivement le n° du jour, le n° du mois et le n° de l'année puis affiche cette date en remplaçant le n° du mois par son équivalent en toute lettre.

Exemple

Si n°jour =1, n° du mois = 5 et n° de l'année =2005 le programme affichera 1 mai 2005.

3. Ecrivez une analyse puis un algorithme d'un problème qui saisit un chiffre et affiche son nom, si la donnée saisie n'est pas un chiffre alors afficher "Ce n'est pas un chiffre".

4.2. Structures de contrôle itératives

On appelle boucle une structure ou un ensemble d'instructions qui sera exécuté plusieurs fois. La boucle permet d'appliquer une opération (c'est-à-dire un ensemble d'instructions) à chaque élément d'une liste. Le nombre de répétitions peut être connu ou inconnu à l'avance. C'est en fonction de ce critère que se fera le choix de la structure itérative. En effet, on distingue deux sortes de structures de contrôle itératives :

1. les boucles dans lesquelles le nombre d'exécutions est connu de prime abord : itérations.
2. Les boucles dans lesquelles le nombre d'exécutions est inconnu au départ : répétitions.

Les structures de contrôle itératives sont de deux types :

- La structure itérative complète ou la structure **POUR ... Faire ...**
- Les structures répétitives à condition d'arrêt, composées de deux structures :
- La structure **REPETER ... Jusqu'à ...**
- La structure **TANT QUE ... Faire ...**

Dans la suite du cours, vous allez vous rappeler et appliquer ces structures dans la résolution de quelques problèmes.

4.2.1. Structure itérative complète

La structure de contrôle itérative complète est utilisée dans le cas où le traitement à faire nécessite une répétition avec le nombre des itérations connu à l'avance. C'est, donc, la structure que l'on utilise lorsqu'on veut répéter plusieurs instructions un nombre connu de fois.

La structure de contrôle itérative complète est représentée comme suit :

Au niveau de l'algorithme	Au niveau de Pascal
<pre> ----- } Initialisation (s) ----- } </pre>	<pre> -----; Initialisation (s) -----; </pre>
<pre> POUR Cp de Vi à Vf FAIRE Instruction 1 Instruction 2 Instruction n FINPOUR </pre>	<pre> FOR Cp := Vi TO Vf DO Begin Instruction 1; Instruction 2; ; ; Instruction n; End; </pre>

Attention:

Evitez que le traitement modifie la valeur du compteur de la boucle.

Les expressions Vi et Vf sont évaluées une fois pour toutes à l'entrée de la boucle. On affecte à Cp la valeur de Vi. Tant que Cp est inférieur ou égal à Vf on effectuera le traitement puis Cp est incrémenté de 1.

Remarques

1. Le compteur de la boucle est de type scalaire.
2. Le pas est par défaut de 1.
3. Si le pas est de (-1), la syntaxe en Pascal devient :

For Cp := Vi Downto Vf do

4. La partie "Initialisations" comporte :
 - i. les initialisations des variables utilisées dans le traitement répétitif.
 - ii. un message dont la suite se trouve au niveau du traitement répétitif
5. Le changement de la valeur du compteur est automatique ; le pas est de 1 ou de -1.

4.2.2. Structures de contrôle itérative à condition d'arrêt

Il existe deux structures de contrôle itératives à condition d'arrêt. Il s'agit de la structure REPETER ... JUSQU'A... et de la structure TANT QUE ... FAIRE Ces structures sont utilisées Lorsque'il s'agit d'un problème répétitif dont on ne connaît pas le nombre de répétitions.

4.2.2.1. Structure REPETER ... JUSQU'A ...

La structure REPETER...JUSQU'A... est utilisée quand il s'agit de répéter un traitement un nombre de fois inconnu à l'avance et qu'on est sûr que le traitement itératif s'exécutera au moins une fois. Le traitement est toujours exécuté au moins une fois. Dès que la condition d'arrêt devient vraie, la boucle est abandonnée et le programme continue en séquence.

Syntaxe

La structure **REPETER ... JUSQU'A ...** est représentée comme suit :

Au niveau de l'algorithme	Au niveau de Pascal
Initialisation (s)	-----;
REPETER	-----;
Instruction 1	Repeat
Instruction 2	Instruction 1;
....	Instruction 2;
....	;
Instruction n	;
JUSQU'A condition(s) d'arrêt	Instruction n;
	Until condition(s) d'arrêt;

Remarques

- 1) La condition d'arrêt est considérée comme une condition de sortie car, une fois elle est vérifiée, on quitte la boucle sinon on itère encore jusqu'à sa vérification (sa valeur est égale à vrai).
- 2) La condition d'arrêt peut être simple ou composée.
- 3) Les caractéristiques de la boucle **REPETER...JUSQU'A** en font une structure adaptée pour le contrôle de la saisie des données. On répète la saisie jusqu'à ce que les contraintes spécifiées dans l'énoncé du problème soient respectées.

Exercices

1. Exécutez manuellement l'algorithme suivant pour chacune des valeurs suivantes de la variable choix :
 - 1) choix = 10
 - 2) choix = 0
 - 3) choix = 1

DEBUT Algorithme saisie

Variables

Choix : Entier

Début action principale

Répéter

Ecrire("Choix = ")

Lire(Choix)

Jusqu'à (choix dans [1..3])

Ecrire(Choix)

Fin action principale**FIN** Saisie

Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « *Ap1* ».

2. Revenons à l'application de la partie « Structure conditionnelle à choix », qui consiste à réaliser une application informatique qui saisit une date en introduisant successivement le n° du jour, le n° du mois et le n° de l'année puis afficher cette date en remplaçant le n° du mois par son nom.

Exemple

Si n° jour = 1, n° du mois = 5 et n° de l'année = 2005 le programme affichera 1 mai 2005.

4.2.2.2. Structure TANT QUE ... FAIRE ...

La structure **TANT QUE...FAIRE...** est utilisée quand il s'agit de répéter un traitement un nombre de fois inconnu à l'avance et que le traitement itératif peut ne pas s'exécuter. Le principe de fonctionnement de cette boucle est simple : Quand le programme atteint la ligne du TANT QUE. Il examine alors la valeur de la condition. Si cette valeur est VRAIE, le programme exécute les instructions qui suivent, jusqu'à ce qu'il rencontre la ligne FIN TANTQUE. Il retourne ensuite sur la ligne du TANTQUE, procède au même examen, et ainsi de suite. Le traitement ne s'arrête que lorsque la condition prend la valeur FAUSSE.

Syntaxe

La structure **TANT QUE ... FAIRE ...** est représentée comme suit :

Au niveau de l'algorithme	Au niveau du Pascal
Initialisation (s)	-----;
TANT QUE condition(s) FAIRE	-----; WHILE condition(s) DO
Instruction 1	Begin
Instruction 2	Instruction 1;
....	Instruction 2;
....	;
Instruction n	;
Fin TANT QUE	Instruction n;
	End;

Remarques :

- 1) La condition d'arrêt est considérée comme une condition d'entrée dans la boucle car, une fois qu'elle n'est pas vérifiée, on quitte la boucle sinon on itère encore jusqu'à sa non vérification.
- 2) La condition d'entrée dans la boucle peut être simple ou composée.

Exercices

1. Exécutez manuellement l'algorithme suivant pour chacune des valeurs suivantes de la variable c :

- 1) c = 7
- 2) c = 4
- 3) c = 5

DEBUT Algorithme choix

Variables

c : Entier

Lire (c)

Tant que (c=5) **Faire**

Si (c MOD 2 = 0) **alors**

 Ecrire(2*c)

Sinon

 Ecrire (c*c)

Fin si

 Lire(c)

Fin Tant Que

FIN Saisie

Traduisez cet algorithme en Pascal et enregistrez le programme source sous le nom « **Ap1** ».

2. On se propose d'écrire un algorithme qui permet de saisir une suite d'entiers qui se termine par -1 puis de calculer et d'afficher la somme de ces valeurs sauf la dernière.

Exemple

Si la suite saisie est 5, 6, 7, -4, -1 la valeur affichée est 14

Chapitre 5 : Etapes de traitement d'informations par un ordinateur

5.1. Entrée et sortie des informations

Les données à traiter et les données des commandes du programme sont conservées dans des fichiers respectivement les fichiers des données et les fichiers-programmes. Un fichier est un ensemble d'informations homogènes structurées en rubriques.

Exemple :

Un fichier des étudiants peut comprendre comme rubrique le numéro nominal, le nom et le post-nom, la date de naissance, la classe. Une ligne d'informations concernant un étudiant dans ce fichier est appelé *enregistrement*.

La fin d'un fichier est marquée par un enregistrement spécial appelé *label de fin de fichier*. Sur la carte perforée, ce label est une suite des caractères /* (se lit slash astérisque).

Les données traitantes (instructions du programme) et les données doivent être introduites dans la mémoire centrale de l'ordinateur. Le programme est introduit une seule fois tandis que les données en entrée accèdent à la mémoire centrale à tour de rôle lorsque l'ordre de lecture du fichier qui les contient est donné. L'enregistrement admis en mémoire centrale pour lecture occupe un emplacement dans la mémoire et ayant sa structure et réservé d'avance par le programmeur, cet emplacement est appelé zone de lecture ou zone d'entrée.

Chaque ordre de lecture efface la zone de lecture avant d'y admettre un enregistrement. D'autres informations en entrée non contenues dans le fichier sont introduites dans la mémoire centrale soit à partir d'un périphérique d'entrée soit par initialisation dans le programme. On réserve pour chacune de ces informations un emplacement en mémoire centrale appelé zone de travail. Il en est de même pour toutes les informations obtenues par calcul en utilisant les données existantes. Les données sont traitées par le programme pour obtenir les données en sortie ou résultats.

Les résultats sont conservés en mémoire centrale à un emplacement appelé zone de sortie ou zone d'écriture que le programmeur doit réserver au préalable. C'est par un ordre d'écriture que les résultats sortent de la mémoire centrale. La zone de sortie est garnie en informations par ordre de transfert. Elle n'est pas nettoyée avant d'y transmettre les informations, c'est pourquoi elle doit être nettoyée par ordre de mise à blanc (MAB).

5.2. Etape de résolution d'un problème de logique de programmation

Pour résoudre avec succès un problème, il importe de bien le cerner en analysant de manière à préciser les informations nécessaires en entrée et le traitement à appliquer à ces informations afin d'obtenir les résultats attendus. Pour cela il faut :

a. Analyser les informations disponibles

Cette analyse consiste à dresser une liste des informations disponibles non contenues dans le fichier pour lesquelles une zone de travail doit être réservée et les informations contenues dans le fichier pour lesquelles on réserve une zone de lecture. Dès que l'on donne un ordre de lecture du fichier en entrée, les informations disponibles contenues dans ce fichier accèdent à la zone de lecture. Un seul enregistrement est admis à la fois, les informations disponibles non contenues dans le fichier accèdent à la mémoire centrale soit en leur affectant des valeurs initiales dans une zone de travail réservée préalablement, soit en les plaçant dans l'instruction qui les utilise.

b. Analyser les résultats attendus

En appliquant divers traitements aux informations disponibles, le programmeur doit produire les résultats attendus sous la forme et la présentation voulue par l'utilisateur. Il s'avère donc nécessaire de dresser une liste d'informations à sortir de la mémoire centrale par ordre d'écriture en distinguant celles qui sont déjà disponibles en mémoire centrale de celles qui ne sont obtenues qu'après traitement (calcul). Une zone de travail doit être réservée par le programmeur pour chaque information calculée.

Pour chaque information en sortie, il importe de préciser à quel nom et à quelle condition la sortir.

c. Application des traitements (calcul, affectation, etc.) aux informations disponibles

Pour chaque information en sortie, il faut :

- Préciser comment l'obtenir en décrivant toutes les opérations nécessaires ;
- Pour chaque opération, préciser à quel moment et à quelle condition l'effectuer.

Certains résultats ne servent que d'intermédiaires aux calculs d'autres résultats.

5.3. Phases

a. Traitement de début

C'est l'ensemble des opérations effectuées avant la lecture des données. Dans cette étape on procède :

- Aux initialisations des zones de travail et à la réservation des zones de lecture et d'écriture ;
- A l'ouverture des fichiers ;
- Aux calculs ne nécessitant pas les informations contenues dans les fichiers.

b. Traitement de chaque enregistrement ou traitement de détail

C'est l'ensemble des opérations situées entre la lecture du fichier et le retour à la lecture du fichier. Cette opération concerne les opérations de calculs, de transfert,

d'écriture, de réinitialisation, etc définies par le programmeur pour être appliquées aux enregistrements.

c. Traitement de fin de fichier

C'est l'ensemble des opérations exécutées lorsque le fichier est à sa fin.

Exemple :

On dispose d'un fichier des ventes « FVTES » contenant pour chaque produit vendu le code du produit (CODPRO), le libellé du produit (LIBPRO), la quantité vendue (QTE) et le prix unitaire de vente (PUV). On demande d'imprimer sur un état appelé LISPRIX le code du produit, le libellé du produit, la quantité vendue et le prix total de chaque produit.

Solution

1. L'analyse des informations disponibles

- Contenues dans le fichier FVTES : CODPRO, LIBPRO, QTE
- Non contenues dans le fichier : -

2. L'analyse des résultats attendus

- Disponibles en mémoire centrale : ZCODPRO, ZLIBPRO, ZQTE
- A calculer : ZPT (Prix total)
- Pour les informations qui sont disponibles dans le fichier, on doit leur réserver une zone de lecture (ZLEC). Pour les informations non contenues dans le fichier, on réserve une zone de travail.
- Pour les informations en sortie on doit leur réserver une zone d'écriture (ZECR)

3. Application de traitement

a. Recherche de ZCODPRO, ZLIBPRO, ZQTE

10. Début

20. Réservation ZLEC, ZECR

30. Ouvrir FVTES, LISPRIX

40. Lire FVTES dans ZLEC

50. Si /* alors aller à 100

60. MAB ZECR

70. Transférer

ZCODPRO ← CODPRO

ZLIBPRO ← LIBPRO

ZQTE ← QTE

80. Ecrire LISPRIX d'après ZECR

90. Aller à 40

100. fermer FVTES, LISPRIX

110. Fin

b. Recherche de ZPT

10. Début

20. Initialisation dans la zone de travail PT à 0

30. Réserver ZLEC, ZECR

40. Ouvrir FVTES, LISPRIX

50. Lire FVTES dans ZLEC

60. Si /* alors aller à 130

70. Calculer $PT \leftarrow QTE * PUV$

80. MAB ZECR

90. Transférer

$ZPT \leftarrow PT$

100. Ecrire LISPRIX d'après ZECR

110. Réinitialiser PT à 0

120. Aller à 50

130. fermer FVTES, LISPRIX

140. Fin

c. Algorithme général

10. Début

20. Initialisation dans la zone de travail PT à 0

30. Réserver ZLEC, ZECR

40. Ouvrir FVTES, LISPRIX

50. Lire FVTES dans ZLEC

60. Si /* alors aller à 130

70. Calculer $PT \leftarrow QTE * PUV$

80. MAB ZECR

90. Transférer

$ZCODPRO \leftarrow CODPRO$

$ZLIBPRO \leftarrow LIBPRO$

$ZQTE \leftarrow QTE$

$ZPT \leftarrow PT$

100. Ecrire LISPRIX d'après ZECR

110. Réinitialiser PT à 0

120. Aller à 50

130. fermer FVTES, LISPRIX

140. Fin

Exercices

1. Un fichier FJRN contient des informations sur les travailleurs journaliers d'une entreprise suivant la structure ci-après : Numéro d'identification (NUMID), nom et post-nom (NOM), code sexe (SEXE) : F pour féminin et M pour masculin, le nombre d'heures prestées (HP) et le taux horaire (TH). Le taux horaire est fixé suivant le sexe et le nombre d'heures prestées.

Pour les femmes ayant totalisé :

- Moins de 20 heures : 3\$/h
- 20 heures : 5\$/h
- Plus de 20 heures : 7\$/h

Pour les hommes ayant totalisé :

- Moins de 20 heures : 5\$/h
- 20 heures : 7\$/h
- Plus de 20 heures : 9\$/h

On demande d'imprimer un état de paie appelé PAIE reprenant les numéros d'identification, les noms, les sexes en clair et le montant à payer à l'agent journalier. Le montant à payer est le produit du taux horaire et du nombre d'heures prestées.

2. On dispose d'un fichier de versement appelé FVERS contenant pour chaque étudiant le numéro matricule (MATR), le nom (NOM) et le montant versé comme frais d'études (MNT). On demande d'imprimer un état appelé LIVERS reprenant pour chaque étudiant le numéro matricule, le nom et le reste à payer. Le reste à payer est calculé sur un total de 150\$. On imprimera à la fin de la liste une ligne reprenant le montant total payé pour tous les étudiants et une ligne reprenant le montant restant à payer ou à percevoir sur l'ensemble des étudiants.
3. Une entreprise désire imprimer un répertoire de ses agents. Elle dispose d'un fichier du personnel appelé FIPERS contenant pour chaque agent le numéro matricule (MATR), le nom (NOM), la date de naissance (DATNAIS), le service (SERV). On demande d'imprimer un état nommé LISAG reprenant pour chaque agent toutes les informations en entrée.

4. Soit un fichier nommé ETUD reprenant les étudiants ayant participé à un concours de sélection en Algorithmique. Le fichier contient les noms des étudiants et les pourcentages des points obtenus par chacun d'eux. On demande :
- De lister ce fichier (éditer une liste LISET)
 - Mettre à côté de l'étudiant qui a moins de 50% la mention médiocre sinon Bon
 - Compter le nombre d'étudiants médiocres et le nombre total d'étudiants

Chapitre 6 : Sous-programmes, fonctions et procédures

6.1. Sous-programmes

Imaginez que nous avons à exécuter le même code plusieurs fois de suites, à des endroits différents du programme, de telle sorte que l'utilisation d'une boucle ou de conditions serait impossible.

On pourrait tout simplement écrire autant de fois que nécessaire le bout de code correspondant. C'est vrai, on pourrait...

Mais on peut faire beaucoup plus malin : créer un sous-programme qui sera appelé lorsqu'il le faudra.

On utilise pour cela l'instruction **Aller à** pour appeler le sous-programme. En BASIC par exemple, cette instruction est traduite par **GOSUB** ou **GOTO**. On spécifie le nom de l'étiquette où se trouve le code à exécuter et le tour est joué !

Pour le cas de BASIC, il faudra tout de même spécifier la fin du sous-programme avec l'instruction **RETURN**. Lorsque le programme rencontrera cette instruction, il reviendra au GOSUB qui l'a appelé. Mais un exemple vaut mieux qu'un long discours :

```
CONST PI = 3.14159265458
CLS
PRINT "Ce programme utilise l'emploi de GOSUB et RETURN pour calculer l'aire
d'un cercle"
PRINT
INPUT "Entrez le rayon du cercle : ", Rayon
GOSUB CalculAire          ' Appel du sous programme 'CalculAire'
PRINT "L'aire de ce cercle est de "; Aire
END

CalculAire:                ' Etiquette 'CalculAire'. C'est le sous-programme
Aire = PI * Rayon ^ 2
RETURN                    ' Fin du sous-programme, on retourne au GOSUB
```

À noter que comme pour GOTO, GOSUB peut aussi se transformer en **ON GOSUB**. Ainsi, on pourra tester la valeur d'une variable et aller au sous-programme approprié.

6.2. Procédures

Mais on peut encore mieux faire. Comment ? En créant une procédure. Par exemple, on pourra remarquer que pour écrire un texte un certain nombre de fois, il faut une instruction de répétition (Pour, Tant que ou Répéter).

Si on doit écrire du texte un peu partout sur l'écran, répéter l'une des instructions répétitives à chaque fois deviendra vite ennuyeux. On peut alors créer une procédure.

Maintenant, on va devoir spécifier les arguments à utiliser. Qu'est-ce qu'un argument ? C'est une valeur qui sera passée en paramètres lors de l'appel de la procédure. Dans notre cas, nos arguments sont :

- Le texte à écrire ;
- Le nombre de fois qu'il faut écrire ce texte.

Pour cela, il faut rajouter les arguments entre parenthèses lors de la définition et de l'appel de la procédure.

Procédure Ecrit(Texte, nombre)

Fin Procédure

Maintenant, nous devons traiter ces arguments que le programme va nous passer quand la procédure sera appelée.

Procédure Ecrit (Texte, nombre)

Pour i ← 1 à nombre faire

 Ecrire Texte

Fin Pour

END SUB

Notre procédure est finie. Nous allons la tester. Il nous faut revenir dans le corps du programme.

Début

Lire T

Lire n

Ecrit(Texte, nombre)

Fin

6.3. Fonctions

Une fonction, c'est exactement la même chose qu'une procédure mais à la seule différence qu'une fonction renvoie une valeur alors qu'une procédure non. D'ailleurs, dans certains langages de programmation, cette différence ne se fait pas.

Par exemple, considérons une fonction que l'on appellera **Aire** qui permet de calculer l'aire d'un cercle de rayon r . Comme pour une procédure, il nous faut spécifier les arguments. Ici, il n'y aura que le rayon, donc :

```
FONCTION Aire (r)
```

```
FIN FONCTION
```

Ensuite, on écrit le code à exécuter, soit :

```
FONCTION Aire (r)
```

```
Aire = 3.14159265458 * r ^ 2
```

```
FIN FONCTION
```

À ce stade là, quelques explications sont peut-être nécessaires.

- Déjà, nous n'avons pas défini de constante pour PI, mais avons utilisé la valeur numérique directement
- Ensuite, on a affecté à *Aire*, qui est le nom de la fonction, le résultat.

Attention ! Ici, on a affecté directement le résultat à Aire. Mais gardez à l'esprit qu'une fonction ne se comporte pas comme une variable. Du fait, on ne peut pas écrire des choses du style :

```
FONCTION Aire (r)
```

```
Aire = 3.14159265458
```

```
Aire = Aire * r
```

```
Aire = Aire * r
```

```
FIN FONCTION
```

La logique est bonne, mais ceci aurait pour effet d'appeler la fonction Aire, qui à son tour appellera la fonction Aire, etc... Si on avait voulu suivre cette logique il aurait fallu écrire :

```
FONCTION Aire (r)
```

```
Resultat = 3.14159265458
```

```
Resultat = Resultat * r
```

```
Resultat = Resultat * r
```

```
Aire = Resultat
```

```
FIN FONCTION
```

Maintenant que notre fonction est définie, nous allons l'utiliser. Pour cela, rentrons dans le corps du programme principal, et écrivons :

```
Début
```

```
Ecrire "Entrez le rayon du cercle : "
```

```
Lire Rayon
```

```
Ecrire "L'aire du cercle est ", Aire(Rayon)
```

```
Fin
```

6.4. Variables dans les fonctions

Une variable qui est utilisé dans une procédure ou une fonction ne sera accessible que dans celle-ci.

C'est-à-dire que si par exemple, vous utilisez une variable x dans une procédure ou une fonction, dans aucun cas vous ne pourrez avoir accès à elle, que ce soit à partir d'une autre procédure ou fonction, ou même du corps principal du programme. On dit alors que cette variable est une variable *locale*. De même, cette variable x va perdre sa valeur entre chaque appel. Essayez le programme suivant (la procédure est placée sur le même listing) :

```
Début  
Lecture  
Lecture  
Fin
```

```
Procédure Lecture  
Ecrire "Entrer une valeur pour x : "  
Lire x  
Ecrire x  
Fin Procédure
```

Dans l'exemple ci-dessus, X perd sa valeur, vous pouvez le voir par vous-même.

Si vous voulez rendre accessible par toutes les procédures et fonctions une variable, il faut la rendre publique. Avec l'exemple suivant, la variable est inaccessible :

```
MaVariable = 5  
MaProcédure  
  
SUB MaProcédure  
PRINT "La valeur de la variable est "; MaVariable  
END SUB
```

Chapitre 7 : Fonctions récursives

7.1. Définition récursive – Algorithme récursif

Une définition récursive est une définition dans laquelle intervient ce que l'on veut définir. Un algorithme est dit récursif lorsqu'il est défini en fonction de lui-même.

Dans le cadre de ce cours, nous ne nous intéresserons qu'aux programmes et algorithmes récursifs. Mais la notion de définition récursive est beaucoup plus générale :

en mathématiques : définition de l'exponentielle :

$$\forall x \in \mathbb{R}; f'(x) = f(x) \text{ et } f(0) = 1.$$

en programmation : définition en QBasic de la fonction exponentielle :

```

FUNCTION Factoriel(n)
  IF n=0 THEN
    Factoriel = 1
  ELSE
    Factoriel = n*Factoriel(n-1)
  END IF
END FUNCTION

```

7.2. Récursivité simple

Revenons à la fonction puissance $x \rightarrow x^n$. Cette fonction peut être définie récursivement :

$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x \times x^{n-1} & \text{si } n \geq 1 \end{cases}$$

L'algorithme correspondant s'écrit :

```

PUISSANCE (x, n)
  Si n = 0 alors
    renvoyer 1
  sinon
    renvoyer x x PUISSANCE(x, n-1)
  Fin Si

```

7.3. Récursivité multiple

Une définition récursive peut contenir plus d'un appel récursif. Nous voulons calculer ici les combinaisons C_n^p en se servant de la relation de Pascal :

$$C_n^p = \begin{cases} 1 & \text{si } p = 0 \text{ ou } p = n \\ C_{n-1}^{p-1} + C_{n-1}^p & \text{sinon} \end{cases}$$

L'algorithme correspondant s'écrit :

```
COMBINAISON (n, p)
  Si p = 0 ou p = n alors
    renvoyer 1
  Sinon
    renvoyer COMBINAISON (n-1, p-1) + COMBINAISON (n-1, p)
  Fin Si
```

7.4. Récursivité mutuelle

Des définitions sont dites *mutuellement récursives* si elles dépendent les unes des autres. Ça peut être le cas pour la définition de la parité :

$$Pair(n) = \begin{cases} vrai & \text{si } n = 0 \\ Impair(n-1) & \text{sinon} \end{cases} \text{ et } Impair(n) = \begin{cases} faux & \text{si } n = 0 \\ Pair(n-1) & \text{sinon} \end{cases}$$

Les algorithmes correspondants s'écrivent :

```
PAIR (n)
  Si n = 0 alors
    renvoyer vrai
  Sinon
    renvoyer IMPAIR (n-1)
  Fin Si
```

et

```
IMPAIR (n)
  Si n = 0 alors
    renvoyer faux
  sinon
    renvoyer PAIR (n-1)
```

7.5. Récursivité imbriquée

La fonction d'Ackermann est définie comme suit :

$$Ack(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ Ack(m - 1, 1) & \text{si } m > 0 \text{ et } n = 0 \\ Ack(m - 1, Ack(m, n - 1)) & \text{sinon} \end{cases}$$

d'où l'algorithme :

ACKERMANN(m, n)

Si $m = 0$ **alors**

renvoyer $n+1$

Sinon

Si $n = 0$ **alors**

renvoyer ACKERMANN($m-1, 1$)

Sinon

Renvoyer ACKERMANN($m-1, ACKERMANN(m, n-1)$)

Fin Si

Fin Si

7.6. Principe et dangers de la récursivité

Principe et intérêt : ce sont les mêmes que ceux de la démonstration par récurrence en mathématiques. On doit avoir :

- un certain nombre de cas dont la résolution est connue, ces « cas simples » formeront les cas d'arrêt de la récursion ;
- un moyen de se ramener d'un cas « compliqué » à un cas « plus simple ».

La récursivité permet d'écrire des algorithmes concis et élégants.

Difficultés :

- la définition peut être dénuée de sens :
Algorithme A(n)
renvoyer A(n)
- il faut être sûrs que l'on retombera toujours sur un cas connu, c'est-à-dire sur un cas d'arrêt ; il nous faut nous assurer que la fonction est complètement définie, c'est-à-dire, qu'elle est définie sur tout son domaine d'applications.

Moyen : existence d'un ordre strict tel que la suite des valeurs successives des arguments invoqués par la définition soit strictement monotone et finit toujours par atteindre une valeur pour laquelle la solution est explicitement définie.

L'algorithme ci-dessous teste si a est un diviseur de b .

DIVISEUR (a, b)

Si $a \leq 0$ **alors**

renvoyer Erreur

sinon

si $a \geq b$ **alors**

$a = b$ (test d'égalité)

sinon

renvoyer DIVISEUR($a, b - a$)

Fin Si

Fin Si

La suite des valeurs $b, b-a, b-2a$, etc. est strictement décroissante, car a est strictement positif, et on finit toujours par aboutir à un couple d'arguments (a, b) tel que $b-a$ est négatif, cas défini explicitement.

Cette méthode ne permet pas de traiter tous les cas :

SYRACUSE(n)

Si $n = 0$ ou $n = 1$ **alors**

renvoyer 1

sinon

si $n \bmod 2 = 0$ **alors**

renvoyer SYRACUSE ($n/2$)

sinon

renvoyer SYRACUSE ($3n+1$)

Fin Si

Fin Si

Problème ouvert : l'algorithme est bien défini et vaut 1 sur $\mathbb{N}$.

Question : N'y a-t-il vraiment aucun moyen de déterminer automatiquement si un algorithme récursif quelconque va terminer ?

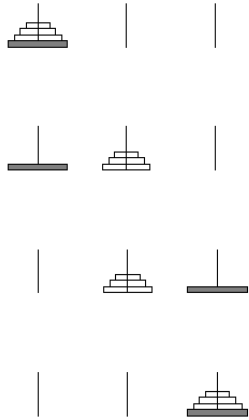
7.7. Exemple d'algorithme récursif : Les tours d'Hanoï

7.7.1. Le problème

Le jeu est constitué d'une plaquette de bois où sont plantées trois tiges. Sur ces tiges sont enfilés des disques de diamètres tous différents. Les seules règles du jeu sont que l'on ne peut déplacer qu'un seul disque à la fois, et qu'il est interdit de poser un disque sur un disque plus petit.

Au début tous les disques sont sur la tige de gauche, et à la fin sur celle de droite.

7.7.2. Résolution



Hypothèse : on suppose que l'on sait résoudre le problème pour $(n - 1)$ disques.

Principe : pour déplacer n disques de la tige A vers la tige C, on déplace les $(n - 1)$ plus petits disques de la tige A vers la tige B, puis on déplace le plus gros disque de la tige A vers la tige C, puis on déplace les $(n - 1)$ plus petits disques de la tige B vers la tige C.

Validité : il n'y a pas de viol des règles possible puisque le plus gros disque est toujours en « bas » d'une tige et que l'hypothèse (de récurrence) nous assure que nous savons déplacer le « bloc » de $(n - 1)$ disques en respectant les règles.

7.7.3. Algorithme

HANOI(n , départ, intermédiaire, destination)

Si $n = 1$ **alors**

déplacer le disque supérieur de la tige départ vers la tige destination

Sinon

HANOI($n-1$, départ, destination, intermédiaire)

déplacer le disque supérieur de la tige départ vers la tige destination

HANOI($n-1$, intermédiaire, départ, destination)

Fin Si

7.7.4. Exécution avec trois disques

1. Déplace un disque de la tige départ vers la tige destination
2. Déplace un disque de la tige départ vers la tige intermédiaire
3. Déplace un disque de la tige destination vers la tige intermédiaire
4. Déplace un disque de la tige départ vers la tige destination
5. Déplace un disque de la tige intermédiaire vers la tige départ
6. Déplace un disque de la tige intermédiaire vers la tige destination
7. Déplace un disque de la tige départ vers la tige destination

Il ne faut pas chercher à comprendre *comment* ça marche, mais *pourquoi* ça marche...

7.8. Dérécursivation

Dérécursiver, c'est transformer un algorithme récursif en un algorithme équivalent ne contenant pas d'appels récursifs.

7.8.1. Récursivité terminale

Un algorithme est dit récursif terminal s'il ne contient aucun traitement après un appel récursif.

Exemple :

ALGORITHME P(U)

si C **alors**

 D

 P($\alpha(U)$)

Sinon

 T

Fin Si

où :

- U est la liste des paramètres ;
- C est une condition portant sur U ;
- D est le traitement de base de l'algorithme (dépendant de U) ;
- $\alpha(U)$ représente la transformation des paramètres ;
- T est le traitement de terminaison (dépendant de U).

Avec ces notations, l'algorithme P équivaut à l'algorithme suivant :

ALGORITHME P'(U)

Tant que C **faire**

 D

$U \leftarrow \alpha(U)$

Fin Tant que

 T

L'algorithme P' est une version **dérécursivée** de l'algorithme P.

7.8.2. Récursivité non terminale

Ici, pour pouvoir dérécuriver, il va falloir sauvegarder le contexte de l'appel récursif, typiquement les paramètres de l'appel engendrant l'appel récursif.

Originellement, l'algorithme est :

ALGORITHME Q(U)

Si $C(U)$ **alors**

$D(U)$

$Q(\alpha(U))$

$F(U)$

Sinon

$T(U)$

Fin Si

Les piles sont des structures de stockage (via les primitives empiler et dépiler) qui fonctionnent sur le principe « le dernier entré est le premier sorti ». Les compilateurs utilisent des piles pour stocker les paramètres des appels de fonctions, et en particulier lors de la transcription des fonctions récursives. Nous mimons ici l'utilisation des piles pour dérécurser l'algorithme.

Après dérécurvation on obtiendra donc :

ALGORITHME Q'(U)

empiler(nouvel_appel, U)

Tant que pile non vide **faire**

dépiler(état, V)

Si état = nouvel_appel **alors**

$U \leftarrow V$

Si $C(U)$ **alors** $D(U)$

empiler(fin, U)

empiler(nouvel_appel, $\alpha(U)$)

Sinon $T(U)$

Fin Si

Si état = fin **alors**

$U \leftarrow V$

$F(U)$

Fin Si

Illustration de la dérécursivation de l'algorithme Q

Exemple d'exécution de Q :

```

Appel Q( $U_0$ )
   $C(U_0)$  vrai
   $D(U_0)$ 
  Appel Q( $\alpha(U_0)$ )
     $C(\alpha(U_0))$  vrai
     $D(\alpha(U_0))$ 
    Appel Q( $\alpha(\alpha(U_0))$ )
       $C(\alpha(\alpha(U_0)))$  faux
       $T(\alpha(\alpha(U_0)))$ 
       $F(\alpha(U_0))$ 
     $F(U_0)$ 

```

Remarques

Les programmes itératifs sont souvent plus efficaces, mais les programmes récursifs sont plus faciles à écrire. Les compilateurs savent, la plupart du temps, reconnaître les appels récursifs terminaux, et ceux-ci n'engendrent pas de surcoût par rapport à la version itérative du même programme.

Il est toujours possible de dérécursiver un algorithme récursif.

7.9. Diviser pour régner

Principe

Nombres d'algorithmes ont une structure récursive : pour résoudre un problème donné, ils s'appellent eux-mêmes récursivement une ou plusieurs fois sur des problèmes très similaires, mais de tailles moindres, résolvent les sous-problèmes de manière récursive puis combinent les résultats pour trouver une solution au problème initial.

Le paradigme « diviser pour régner » donne lieu à trois étapes à chaque niveau de récursivité :

- **Diviser** le problème en un certain nombre de sous-problèmes ;
- **Régner** sur les sous-problèmes en les résolvant récursivement ou, si la taille d'un sous-problème est assez réduite, le résoudre directement ;

Références Bibliographiques

- Aho A., Hopcroft J. E. et Ullman J. D., 1975. *The design and analysis of Algorithms*, Addison-wesley, Reading, MA.
- Canevet, D., 2005. *L'algorithmique et le PASCAL*, Ed. Delagrave, Paris.
- Carey M.R. & Johnson D.S., 1979. *Computers & Interactability: a guide to the Theory of the NP-Completeness*.
- Cormen T., Leiserson C.E. et Rivest R.L., 1994. *Introduction to algorithms*, MIT Press.
- Cottet-Emard, F. et Goetgheluck, F., 1993. *Mathématiques sur ordinateur*, Ed. DeBoeck-Wesmael, Bruxelles.
- Courtin J. & Kowarski I., 1998. *Initiation à l'algorithmique et aux structures de données*, Volume 1, Dunod, 2^{ème} édition.
- Knuth D. E., 1968. *The art of Computer programming*. Volume 1: Fundamental Algorithms, Addison-Wesley, Reading, MA.
- Knuth D. E., 1969. *The art of Computer programming*. Volume 2: Seminumerical Algorithms, Addison-Wesley, Reading, MA.
- Knuth D. E., 1971. Mathematical Analysis of algorithms, *Information Processing 71*, Proceedings of the IFIP Congress, Ljubljana, 1971, 19-27.
- Knuth D. E., 1973. *The art of Computer programming*. Volume 3: Sorting and Searching, Addison-Wesley, Reading, MA.
- Knuth D. E., 1976. *Big Omicron and big Omega and big Theta*, SIGACT News, Avril-juin 1976, 18-24.
- Maysonnave J., 1996. *Introduction à l'algorithmique générale et numérique*, Masson, Paris.
- Papadimitriou C.H. & Steiglitz K., 1982. *Combinatorial Optimization: Algorithms & Complexity*.
- Sedgewick R. et Flajolet P., 1996. *Introduction à l'analyse des algorithmes*, International Thomson Publishing France, Paris.

Table des Matières

Chapitre 1 : Généralités sur l'Algorithmique.....	1
1.1. Introduction.....	1
1.2. Qu'est-ce-que c'est que l'Algorithmique ?.....	1
1.3. Problèmes fondamentaux en Algorithmique.....	7
1.3.1. Complexité.....	7
1.3.2. Calculabilité.....	8
1.3.3. Correction	8
1.4. Notion d'Algorithme.....	8
1.4.1. Qu'est-ce qu'un algorithme ?.....	8
1.4.2. Exemple d'un algorithme.....	9
1.4.3. Différences entre algorithmes et programmes	10
1.4.4. Rôle privilégié des ordinateurs	11
1.4.5. Structures de données.....	11
1.4.6. Conventions en algorithmique	12
1.4.7. Quelques conseils pour être bon en algorithmique	13
1.4.8. Sémantique et Syntaxe des algorithmes.....	14
1.4.9. Caractéristiques d'un algorithme	14
1.4.10. Erreurs en programmation	14
1.5. Ordinogramme	15
1.5.1. Symboles d'ordinogramme	15
1.5.2. Règles de graphisme	16
Chapitre 2 : Démarche de résolution de problèmes	18
2.1. Première étape : Position du problème	19
2.2. Deuxième étape : Spécification et analyse du problème.....	20
2.2.1. Exemple du calcul de la surface d'un rectangle.....	20
2.2.2. Solution par les deux approches.....	20
2.3. Troisième étape : Écriture de l'algorithme	22
2.4. Quatrième étape : Ecriture du programme	23
2.5. Cinquième étape : Exécution et tests du programme	26
Chapitre 3 : Les structures de données et les structures algorithmiques simples.....	27

3.1.	Introduction au troisième chapitre	27
3.2.	Constantes et variables.....	27
3.2.1.	Constantes	27
3.2.2.	Variables	28
3.3.	Opérateurs.....	29
3.3.1.	Opérateurs Arithmétiques	29
3.3.2.	Opérateurs relationnels	30
3.3.3.	Opérateurs logiques.....	30
3.3.4.	Fonctions standards.....	30
3.4.	Structures algorithmiques simples	32
3.4.1.	Opération d'entrée.....	33
3.4.2.	Opération de sortie	33
3.4.3.	Opération d'affectation	34
3.5.	Types énumérés	38
3.5.1.	Exemple	39
3.5.2.	Déclaration.....	39
3.5.3.	Fonctions prédéfinies sur les types énumérés	39
3.6.	Types utilisateurs	41
3.6.1.	Déclaration.....	41
3.6.2.	Remarque	41
3.7.	Tableaux.....	41
3.7.1.	Déclaration.....	42
3.7.2.	Tableaux à deux dimensions	43
Chapitre 4 : Les structures algorithmiques de contrôle.....		46
4.1.	Structures de contrôle conditionnelles	46
4.1.1.	Structure conditionnelle simple.....	46
4.1.2.	Structure conditionnelle généralisée	50
4.1.3.	Structure conditionnelle à choix	52
4.2.	Structures de contrôle itératives.....	54
4.2.1.	Structure itérative complète	54
4.2.2.	Structures de contrôle itérative à condition d'arrêt	55
Chapitre 5 : Etapes de traitement d'informations par un ordinateur.....		60
5.1.	Entrée et sortie des informations.....	60

5.2.	Etape de résolution d'un problème de logique de programmation	61
5.3.	Phases.....	62
Chapitre 6 : Sous-programmes, fonctions et procédures		69
6.1.	Sous-programmes	69
6.2.	Procédures.....	70
6.3.	Fonctions.....	70
6.4.	Variables dans les fonctions.....	72
Chapitre 7 : Fonctions récursives.....		73
7.1.	Définition récursive – Algorithme récursif.....	73
7.2.	Récursivité simple.....	73
7.3.	Récursivité multiple	74
7.4.	Récursivité mutuelle	74
7.5.	Récursivité imbriquée	75
7.6.	Principe et dangers de la récursivité	75
7.7.	Exemple d'algorithme récursif : Les tours d'Hanoï.....	76
7.7.1.	Le problème	76
7.7.2.	Résolution	77
7.7.3.	Algorithme	77
7.7.4.	Exécution avec trois disques	77
7.8.	Dérécursivation	78
7.8.1.	Récursivité terminale	78
7.8.2.	Récursivité non terminale	78
7.9.	Diviser pour régner	80
Références Bibliographiques		81
Table des Matières		82